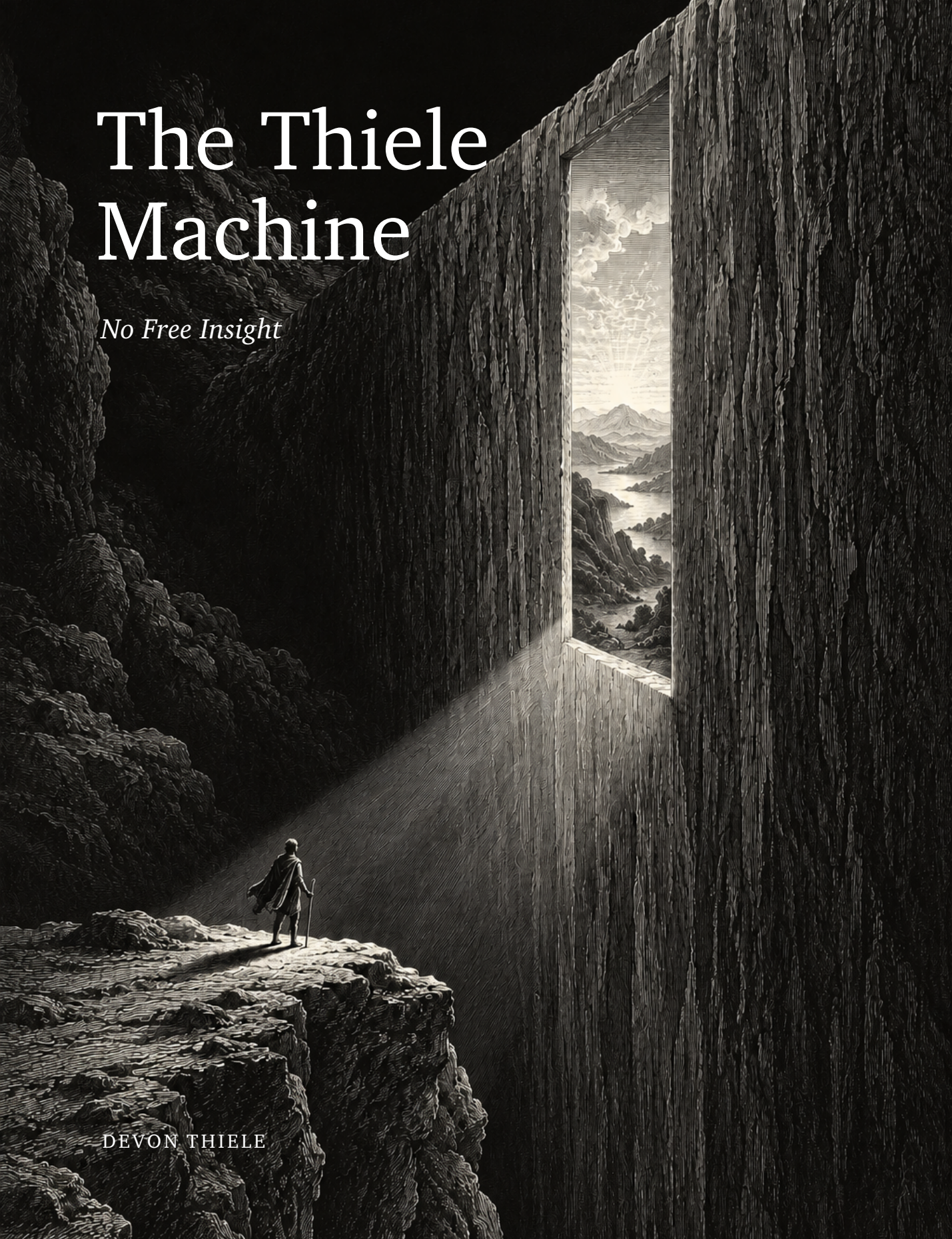
The Thiele Machine

No Free Insight



DEVON THIELE

DEVON THIELE

The Thiele Machine

No Free Insight

The Thiele Machine: No Free Insight

Devon Thiele

Copyright 2025–2026 Devon Thiele. Licensed under Creative Commons Attribution-ShareAlike 4.0 (CC BY-SA 4.0).

The source of this book, its proofs and its build are public: github.com/sethirus/The-Thiele-Machine. Archived at doi.org/10.5281/zenodo.17316437.

Set in Charter. Built from its source with pdfTeX.

The cover and part illustrations were generated with AI in the engraving tradition of Gustave Doré. They are new illustrations, not reproductions of his work.

For Julie, and for my dad.

Julie,

Three years ago, a few months into the rendering engine, I came out of my office and told you I thought I was onto something. You said okay. You didn't ask what it was, or what it was for, or when it would be done, and you didn't ask me for anything I could have shown you, because there was nothing to show. There was a program that had done something it shouldn't have been able to do, and a man who didn't yet know what a Turing machine was.

You gave me the three years anyway. The nights after the lot closed. The weekends. The hours I was in the house and not in the house, because my head was in a wall I couldn't get past. Every dead end, and there were more of those than there are pages here. You never once asked me to stop, and you never once said it was fine when it wasn't. You just kept giving me the next evening.

This book is what I did with them. It's the one thing I've built from nothing that I'm sure of, because I threw out every piece of it I couldn't prove, and what's left holds. I know that isn't what you gave the years for. You gave them for me. But it's what I have to show, and I wanted it to be worth showing.

Dad,

The name on the cover was yours first. I asked you before I used it and you said go ahead. You and Mom taught me that nothing is true just because somebody says it is. This book is that lesson, hundreds of pages long. I put our name on it because I wanted one thing in the world with that name on it that nobody handed me, that I couldn't have faked, and that I can stand behind in front of anyone. Here it is.

Julie carries the name now too. So the machine has it third. This is for the two of you who had it first.

Contents

What this is 8

1 Any machine that steps 11
- 1.1 Is the pencil thing a computer? 11
- 1.2 How far does that go? 12
- 1.3 Try to break it 13
- 1.4 Now ask it about keeping 14
- 1.5 The machine that stops 14
- 1.6 What Turing actually proved 15

2 Running out of room 17
- 2.1 Four people, three chairs 17
- 2.2 This is very old 18
- 2.3 The hotel that never runs out 20
- 2.4 Cut the strip 21
- 2.5 How long can a finite machine hold off? 22
- 2.6 What's being counted 23

3 Why charge for it 24
- 3.1 A rule you can refuse 24
- 3.2 The first door: endless paper 25
- 3.3 The second door: let it come back down 25
- 3.4 The third door: don't charge for it 26
- 3.5 How much 27
- 3.6 Found, assumed, chosen 28

4 The smallest one 30
- 4.1 Clay, seals, and seven witnesses 30
- 4.2 Two columns and a stamp 31
- 4.3 Closer to now 32
- 4.4 Five jobs 33
- 4.5 Two boxes and a coin 34
- 4.6 What the tally is and isn't 35
- 4.7 Any machine can wear one 36
- 4.8 Which question gets charged? 36
- 4.9 The cheapest price list 37

5 More settled 38
- 5.1 A row of cups 38

5.2 Naming what's on the table 40
5.3 The toll on the whole direction 41
5.4 One cup isn't enough 42
5.5 Somebody got here first, from the side 43
5.6 Why crossing a line has to cost 43

6 Looking isn't keeping 45
6.1 A small being at a small door 45
6.2 Bennett moves where the cost is 46
6.3 The demon on a sheet of paper 47
6.4 Two rows of dots 48
6.5 Who holds the pen 49
6.6 Learning after the first look 50
6.7 Finishing together 50
6.8 The certainty count 51
6.9 Who gets charged is partly bookkeeping 51
6.10 Somebody got there first, goddamn it 52
6.11 What the toll is for, said at last 53

7 Looking through a window 55
7.1 Two boxes under a camera 55
7.2 A cave, and the people who got good at it 55
7.3 The shadow of a cube 56
7.4 The whole fact 57
7.5 When does one exist? 57
7.6 People who found this on their own 58
7.7 Notes passed across a room 59
7.8 Slips handed out at desks 60
7.9 Moves that keep to the tabletop 61
7.10 The clerk with two photographs 62
7.11 Look before the concrete covers it 63
7.12 Three ways past the window 63
7.13 What gets through 66

8 Established 67
8.1 Two computers that end up the same 67
8.2 Look harder 68
8.3 So keep the trace 68
8.4 More boxes to tick 70
8.5 Computed, stored, established 71
8.6 The clerk who forgets everything 71
8.7 Information doesn't have it either 72

8.8 Statues that walk off 73
8.9 What "could have failed" means 74
8.10 Computing already had a word for it 75
8.11 The two computers, put back together 77
8.12 What putting it in the step buys 78

What this is

I think in shapes, so I'll start with one.

Take a thing. Anything. A coin on a table, a light switch, a page in a notebook. Right now it's one way. It could be another.

Now do something to it. Flip the coin, hit the switch, write on the page. That's a move: a direction that takes the thing from how it is to how it is next.

Here's a computer you can run with a pencil. Write 3 on a scrap of paper. The move is: erase the number and write the next one up. Do it twice. 3, 4, 5. You just added 2 and 3. Nothing happened there but a thing, a move, and the next thing. Hold on to that picture; whether it covers every computer is something I'll have to earn, and I will, later, with you watching.

Here's the part I couldn't let go of. Some moves just shuffle. Flip the coin and you can flip it back, and nothing is different for good. But some moves make a difference that stays. Write a word in pen. Blow a fuse. Check an answer and mark it checked. After that move something is the case that wasn't before, and it stays the case.

I figured there had to be a cost in that step. Not in the shuffling. In the step where something goes from not settled to settled and stays there. That was a hunch, so test it on the pencil computer.

Tell it when to stop: count up, and when you reach 5, stop. Ask one question of each number: is it done? 3, no. 4, no. 5, yes. And once it says yes it keeps saying yes, because the machine has stopped.

A machine that has stopped is still somewhere on the next tick. It stays where it is: 5 goes to 5. Draw the arrows. 4 goes to 5. 5 goes to 5. Two different starting places, one landing place. Standing on 5, you can't tell whether you just got there or have been there all along.

Could you dodge that? Try letting the stopped machine rock back and forth instead of sitting still: 5-tick, 5-tock, 5-tick, both of them saying done. Draw the arrows. 4 goes to 5-tick. 5-tock goes to 5-tick. Two arrows land on 5-tick, the spot where you came in. Rocking only moved the place where they meet. The one way out is to never land anywhere twice: make every tick write something new, a tally mark, the time, anything, so the paper never repeats. That works as long as there's paper. On a finite sheet you run out, and two arrows land on one spot again.

So whenever this machine settles its answer and keeps it, something gives. Either two states become one, or it uses up fresh paper on every tick for as long as it runs.

That part isn't a hunch. You drew it. You drew it for one small counter; the book proves it for any machine that goes from one state to a next, and shows you where.

What I do with it is a choice, and I'll say so: I charge for the move where the answer goes from not done to done. Charging for it is up to me. That something has to give once it makes that move isn't. The two arrows don't always meet on that very move. Give done two spots, 5a and 5b: 4 goes to 5a, 5a goes to 5b, and 5b stays put. Now the arrows from 5a and 5b meet on 5b, a tick after the answer settled. But after the move from not done to done, there's no avoiding it: two arrows meet somewhere, or the paper never repeats.

Two more things have to be pinned down before the rule means anything, and you can see both with what's already on the table.

First, take the coin again. Flip it, flip it back, forever, free. Ask whether it's settled. It never is. A rule that says "a move that settles something has to cost" never asks the coin for anything, because the coin never settles anything. So the rule alone would let a coin that does nothing count. The machine has to actually settle something, at least once.

Second, suppose a later move could pay you back. The move that settles charges you one, the next move hands you one back, and the whole run comes out free. So prices can be nothing or something, never a refund.

One last thing before the whole definition. Settled doesn't have to be just yes or no. Take a checklist with three boxes. None ticked, one, two, all three: each tick settles it a little more, and ticks don't come off. "Done or not" is the smallest case, a checklist with one box. So when I say "more settled" from here on, picture the checklist.

So here's the Thiele Machine, all of it. A thing. Moves that take it to the next thing. A way to tell how settled the thing is, where settled never goes back to unsettled. Some run, from a start you pick, that actually settles it more than it was. A price on every move, nothing or more, never a refund. And one rule: a move that settles something more than it was costs something.

That's the whole definition. The rest of the book works out what follows from it, one step at a time, and tries to break it.

Now look back at the pencil computer with a stop in it. That's Turing's machine, in miniature. Turing's machine, from his 1936 paper, is a tape and a table of rules: from where it is, the table says where it goes next. A thing and moves. It already has something that settles and stays settled: whether it has stopped, the same question you asked of 3, 4 and 5. What Turing never put in is a price on the move that stops it. Put one in, and a Turing machine that ever stops fits every line above. So this isn't a rival to Turing's machine, and it computes nothing his can't. His machine is one kind of Thiele machine, with the price left out.

Now try this. Give two people the same messy list of numbers. The first sorts it, then goes back down it checking that every number is no bigger than the next. The second copies a sorted list off somebody else's sheet. Now look at the two sheets. Same numbers, same order. Look as hard as you like: nothing on either sheet tells you which one was checked. The difference was real, and neither sheet kept it. It's the same thing you saw standing on 5: you can see where you are, and not how you got there.

So put a box on each sheet that says checked. A plain Thiele machine charges for ticking the box, but nothing in the rules above stops the copier from paying and ticking it anyway. Paying for the tick doesn't make it true. So add one more condition: the box can only be ticked right after a check of that very list passes. Now the copier's tick is refused, and a ticked box tells you a check of that list passed. I call a box like that established. It's a condition you put on the machine. The bare definition doesn't hand it to you.

It's called the Thiele Machine. The afterword says why.

CHAPTER 1

Any machine that steps

When I started all this I didn't know what a Turing machine was. I knew it had something to do with computers, and that's it. I had a program that had done something I didn't expect, and a hunch, and no words for either. Most of the words in this book I found the same way: I'd hit a wall, go looking for whoever had been there before me, and not move on until I could watch the idea actually work. This chapter is the first of those trips. I'll take you on it the way I took it.

The question I had was the one I left you with on page one. I'd been calling that pencil-and-scrap-of-paper thing a computer. Is it? Or did I just make it small enough to look like one?

1.1 Is the pencil thing a computer?

Start with what the word used to mean. Before about 1940 a computer wasn't a machine. It was a job. In the late 1800s the Harvard Observatory hired rooms full of women to do the arithmetic of the sky, cataloguing stars by hand for a fraction of what a man was paid. Some of them found things nobody else had; Henrietta Leavitt worked out a way to tell how far away a star is. (I got that from Dava Sobel's history of those women and from the historians at the American Institute of Physics.) Rooms like that worked out artillery tables in both world wars; that one's from David Alan Grier's history of human computers.

So the pencil thing wasn't a toy version of a computer. For a long time it was the only kind there was. Good start, but that's history, not proof. A person with a pencil can do a lot of things that aren't following rules. I wanted to know what was left if you took the person away and kept only the rules.

That's exactly the question Alan Turing asked, in a paper in 1936. Going in, all I knew was that it had something to do with computers. What's in there is a clerk. He wrote down, as plainly as he could, everything a person sitting at a desk with a pencil does when they follow a procedure, and threw out everything else. What was left had four parts, and you can build it on the table in front of you right now.

Draw a strip of three squares. That's the tape. Write 1 in the first square, 1 in the second, leave the third blank.

Put a coin on the first square. That's where the clerk is looking. Turing called it the head.

On a scrap of paper write the letter A. That's which step of the procedure the clerk is on. Turing called it the state.

Now the rules. Write these two on a card:

> If you're on A and looking at a 1: leave it, move the coin one square right, stay on A.
>
> If you're on A and looking at a blank: write a 1 there, don't move, and change the letter to STOP.

That's the whole machine: a tape, a head, a state, and a card of rules. Now run it. Each tick, read the card and do what it says. After each tick, write down everything about the table: what's on each square, where the coin is, and the letter. I'll put brackets around the square the coin is on.

[1]	1	␣	A
1	**[1]**	␣	A
1	1	**[␣]**	A
1	1	**[1]**	STOP

It started with two 1s and ended with three. It added one. That's the same thing the pencil computer did when it went from 4 to 5.

Now look at what you wrote down on each line. The whole tape, where the coin is, and the letter. Is there anything about this machine at that moment that isn't on the line? Try to think of something. The rules don't change. The tape, the coin and the letter are all there is. So one line is the whole thing, and the card takes one line to the next line. Thing, move, next thing.

So the pencil computer wasn't a cheat. Turing's machine is the pencil computer with a bigger piece of paper.

1.2 How far does that go?

It would be nice if the answer were "every computer, done." It isn't quite. Push on it and it came apart into three different claims that people tend to run together. Here they are, and what each one stands on.

The first one is the one we just did. Every machine you can write down as a whole state and a rule for the next state has the shape from page one. You checked that yourself for Turing's machine: one line, one card, next line.

The second is that the other ways people have written computation down, with registers instead of a tape, or with functions handed to functions (that's Alonzo Church's lambda calculus, from the same year as Turing's paper), compute exactly the same things Turing's machine does. I didn't prove that, and I won't pretend I did. Other people proved it, one pairing at a time; Turing himself sketched the

match with Church's version in an appendix to his 1936 paper. What you can check yourself is that each of those is also a whole state and a rule for the next one, the same way you just did with the coin.

The third is the big one, and here's where I had to be careful. Does this cover everything a person could ever do by following rules? Does it cover every machine anybody could ever build out of real stuff? Those turned out to be claims nobody can prove, and for a good reason. "Everything a person could do by following rules" isn't a precise enough thing to prove anything about. That claim has a name, the Church-Turing thesis. "Every machine you could build" is a claim about physics, and the only way to knock it down would be to build one that beats Turing's, which nobody has done. Neither is a theorem, and in this book neither will get treated like one.

So the debt from page one comes out like this. For every machine you can write down as a whole state and a rule for the next state, paid: you just checked it. For every machine anybody could ever build, that's a claim about the world, it's labeled as a claim, and I'm not going to lean on it.

1.3 Try to break it

Now the question that actually mattered to me. Page one showed that when the counter settled on 5, two arrows had to meet, unless it used fresh paper forever. That was one little counter. Is it true of every machine with this shape?

Don't take my word for it. Get a sheet of paper and do this.

Draw some dots. Ten, twenty, as many as you like. Those are states. From every dot, draw exactly one arrow to some dot. It can point anywhere, even back at the dot it starts from. That's the one rule: every state has exactly one next state.

Now pick a dot to start on and follow the arrows. Keep a list of every dot you've been on.

Do it a few times, with different drawings and different starts. Watch what happens at each step. You'll find that each step does one of three things.

It lands on a dot that isn't on your list yet. A new one.

It lands on the dot you started from. Now you're going around in a circle, and since every dot has only one arrow out, you'll go around that same circle forever.

Or it lands on some other dot that's already on your list. Look at the first time that happens. The first time you got to that dot, you came in on an arrow from the dot right before it on your list. Now you've come in again, on an arrow from the dot you were just on. Those have to be two different dots, because until this step you hadn't been on any dot twice. Two arrows, from two different dots, landing on one. That's a merge, the same thing as 4 and 5 both going to 5. And from there you're on a path you've walked before, so you walk it again, forever.

Now try to make it do a fourth thing. Really try. Draw whatever you want.

Here's why you can't. Wherever the next arrow points, that dot is either on your list or it isn't. If it isn't, it's new. If it is, it's either the start or it isn't the start. There's no other place for the arrow to point. Three cases, and they cover everything.

And notice what you never had to know: how many dots there were. Ten dots or a billion or an endless sheet, it's the same three cases. What you did need was that every dot has exactly one arrow out. Hold on to that, because it's about to matter.

1.4 Now ask it about keeping

Go back to your dots. Color the dot you start on white. Pick some other dots and color them black, and make one rule about the coloring: no arrow is allowed to go from a black dot to a white one. Once you're on black, you stay on black. That's "settled stays settled." White is not done, black is done.

Follow a run from your white start until it lands on black. Now which of the three things can happen after that?

It can't circle back to the start. The start is white, and you can never get back to white once you're on black.

So it's one of the other two. Either it keeps landing on new dots forever, or two arrows meet somewhere.

Try to draw a counterexample: a run that goes from white to black, stays black, never lands on a new dot forever, and never has two arrows meet. You'll be drawing for a while. There isn't one, and the three cases above are the reason.

That's what page one promised: "the book proves it for any machine that goes from one state to a next." You drew it for one counter there. Now you've drawn it for any machine at all, any number of dots.

What it doesn't tell you is anything about heat. Whether two arrows meeting gives off heat in a real device is a question about the device. People argue about it, and they get their own chapter. This is dots and arrows and nothing else.

1.5 The machine that stops

Here's where the "exactly one arrow out" part bites.

Go back to the tape and the coin. The last line was 1 1 [1], STOP. What does the card say to do next? Nothing. There's no rule for STOP. The machine just stops. In dots and arrows: the STOP dot has no arrow out.

That breaks the one thing the three cases needed. So try the keeping question on a machine that stops. Draw two dots. "Working" is white. "Stopped" is black. One arrow, from working to stopped, and no arrow out of stopped.

Follow it. White, then black, then nowhere. Did two arrows meet? No, there's only one arrow. Did it use up new dots forever? No, it stopped. The answer settled and nothing gave.

Now ask it the obvious question: what does a stopped machine do on the next tick? Not "nothing happens," because something is still there. The usual answer is that it stays where it is. So give stopped an arrow back to itself. Now look again. Working points to stopped. Stopped points to stopped. Two arrows, one dot. The merge is back.

What if you don't want it to stay put? Point stopped at a new dot, "parked," and point parked at itself. Now stopped and parked both point at parked. Two arrows, one dot again. The merge moved one dot over. It didn't go away. That's the rocking machine from page one.

So a machine that stops only dodges this by not answering what happens next. Answer it any way you like, and you're back in the three cases: either two arrows meet, or you draw new dots forever.

1.6 What Turing actually proved

Almost every account you'll find says Turing proved the famous halting problem: that no machine can look at every other machine and say whether it stops. Since the stopping machine is right in the middle of this chapter, I went and looked at what he actually wrote, with an AI walking me through the paper and the links it turned up, and it isn't quite what happened.

Turing's own machines in 1936 weren't built to stop at all. The ones he cared about printed out the digits of a number, forever. What he proved no machine could always decide was a question about those: whether a machine keeps printing digits, and whether it ever prints a given symbol. The second one is a few lines away from the halting problem, so in a real sense he had it. But he never wrote it down that way. Stephen Kleene stated it in 1952. Martin Davis seems to be the first to have called it "the halting problem," in 1958. I found that in a paper where Joel David Hamkins and Theodor Nenu went back to Turing's 1936 paper and checked, line by line.

So the machine that stops, the one sitting right in the hole, is Turing's machine the way people later learned to read it. It's the one page one meant when it said Turing's machine already has something that settles and stays settled: whether it has stopped.

Put it together. Read a Turing machine through "has it stopped?" Give the stopped line a next line by letting it stay put. Then the step that stops it is a step where two arrows meet, which you just drew. That's the step I charge for. Charging for it is still my choice. That something gives there isn't.

The dots in this chapter could go on forever. The next question is what happens when they can't, when the paper runs out.

CHAPTER 2

Running out of room

The last chapter left one door open. A run that settles its answer and keeps it either has two arrows meet somewhere, or it lands on a new dot every step, forever. Endless fresh paper. On a sheet that never ends, that's a real way out.

But nobody has a sheet that never ends. So what happens when the paper is finite?

2.1 Four people, three chairs

Do this one with real chairs, or with coins on a table. It takes about a minute.

Draw a box on the floor and call it the done room. Put three chairs inside it and one chair outside, by the door. Sit four people down, one per chair. The people are the states the machine could be in. The three inside are already done. The one by the door isn't.

One move of the machine moves everybody at once: each person gets up and goes to whatever chair the rule sends them to. There are two rules about where they can go.

Nobody inside may end up outside. Done stays done.

The person by the door has to end up inside. This is the move that settles their answer.

Now try to finish the move with one person in each chair.

You can't. The three inside have to stay inside, so they fill the three inside chairs between them. The person from the door has to come in too, and there's no fourth chair. Somebody has to share. Two people, one chair: two arrows, one landing place.

Try it with more chairs. Ten inside, one by the door. Eleven people have to fit into ten chairs. A thousand inside, one by the door: a thousand and one people, a thousand chairs. It doesn't matter how big the room is, as long as it has a size. The insiders take every inside chair, and the newcomer has nowhere to go but somebody's lap.

Notice who ends up sharing. It doesn't have to be the newcomer. The rule could send the newcomer to an empty-looking chair and send two insiders to the same

chair somewhere else. Page one already showed you that: 4 went to 5a, and the arrows met a tick later on 5b. What's forced is that two arrows meet somewhere in that move. Where they meet depends on the rule.

Two words I'll need from here on. A move is one-to-one when two different starting places always end up in two different places: nobody shares a chair. A move that isn't one-to-one is a merge: two arrows, one landing place.

Now say what the chairs showed in those words. Take any move that keeps done states done and takes one state from not done to done, on a machine with K done states. If the move were one-to-one, the K people already inside would fill all K inside chairs with nobody sharing, and the newcomer would have nowhere to land. So it can't be one-to-one. It merges.

That's the whole argument, and it's the chairs. The only thing finiteness did was make "it fills all K done states" true: K people into K chairs with no sharing leaves no empty chair. That's the one place the size of the room mattered.

2.2 This is very old

The piece of mathematics doing the work there is old, and it isn't mine. Put more things into boxes than there are boxes, and some box gets two. People call it the pigeonhole principle. The earliest use of it I could find is Jean Leurechon in 1622, arguing that somewhere two people must have exactly the same number of hairs on their heads. The first name for it I could find came from Dirichlet, who used it in 1842 and later called it the drawer principle. It's the kind of fact a child accepts and a mathematician spends a whole career finding new uses for.

And the use I just made of it turned out to be old too. I found that out twice, and the second time stung more.

The first time was physics. In 1876 Josef Loschmidt pointed out, against Ludwig Boltzmann's account of how a gas settles down, that the laws of motion run just as well backwards: reverse every molecule and the gas retraces its steps. Twenty years later, in 1896, a young assistant of Max Planck's in Berlin pushed harder. He used Henri Poincaré's recurrence theorem from 1890, which says a system like that comes back as close as you like to where it started, again and again, to argue that nothing in it can climb forever. Give it time and it comes back down. The assistant was Ernst Zermelo, the same Zermelo who would put set theory on firm axioms twelve years later. Boltzmann answered the same year, not very politely, that you'd have to wait so long for a return that nobody would ever see one. (Jos Uffink, who wrote the Stanford Encyclopedia's entry on Boltzmann's physics, calls that reply drenched in sarcasm and bitterness, and calls Zermelo's paper a careful logical argument.)

You can see Zermelo's point on the dots from the last chapter. Draw a finite number of dots where no two arrows ever land on the same dot. Every dot has one

arrow in and one arrow out. Follow any run. Can it ever reach a dot it's been on before, other than the start? No: that would take two arrows into one dot. So since the dots are finite and it can't keep finding new ones, it has to come back to its start. Every run goes around in a loop. Now color the start white and some dots black, done stays done. Can a run go from white to black? If it did, it would have to come back around to its white start, and black can't go back to white. So it can't. A machine where no two arrows ever meet, on a finite sheet, can never settle anything new for good.

That's Zermelo's objection, with the infinities taken out. On a finite set you don't need Poincaré's theorem. And the wait isn't enormous; it's at most as many steps as there are dots. Zermelo was aiming it at the second law of thermodynamics, and settling an answer never came into it. A quantum version came in 1957, when P. Bocchieri and A. Loinger proved that a closed quantum system comes back close to where it started. That's a neighbour, not the same proof: even a small quantum system has endlessly many states, and you'd have to say how a record is read off it.

The second time was computer science, and that's the one that stung. Picture a small machine that reads letters, one move per letter. If every letter just reshuffles its states, no two arrows meeting, people call it a permutation automaton, and the sets of inputs such machines can pick out are called group languages. Jean-Éric Pin's 1992 paper on reversible automata sorts out exactly what those machines can recognise, and in that world the chairs argument is folklore: a machine like that can't recognise "has seen a b at some point," because recognising it would mean a done that stays done. Going quantum doesn't get around it. Alex Brodsky and Nicholas Pippenger showed in 2002 that a quantum version that only measures once, at the end, recognises exactly the same group languages. One that's allowed to measure after every letter, and stop on what it sees, does better, and "has seen a b" is one of the things it picks up. A measurement that can stop the machine is a record that stays. That's the extra thing that lets it in.

Then the closest one, and it's recent. In 2023 Thomas Ouldridge and David Wolpert worked out the physics of a small machine built the way real computers are: the same physical step run over and over. They sorted what such machines recognise into two kinds. If the smallest machine for a job has a letter where two arrows meet, every machine for that job does, and on their derivation running it gives off entropy that no cleverer design can get back. If the smallest machine only reshuffles, they find it can run with no unavoidable entropy at all. One of their worked examples is exactly "has not seen a b," which is "has seen a b" turned inside out. So the counting, and a physical price on it, were in their paper three years before this book. Their price comes out of the physics of the device. Mine, in the next chapter, is a choice I make and say so.

There's a cousin that speaks a different language. Constructor theory is David Deutsch and Chiara Marletto's way of stating physics as which transformations a device that runs in a cycle can and can't bring about, and in it a task is irreversible when it's possible and its reverse isn't. Marletto, Vlatko Vedral and their coauthors in 2022 built an irreversible task out of a machine of quantum bits that swap a little with each other, and ran it with photons. Maria Violaris and Marletto argued the same year that this machine, run in a cycle, can turn a pure quantum bit into a mixed one and never back, even though every piece of it can be reversed, and that this suggests erasing may cost more than the usual bound. The chairs are a small, exact, classical relative of that: a set of states the step can't leave, a task that's possible, and no way back. Theirs is quantum and approximate and aimed at physics. Mine is counted. Deutsch and Marletto's paper on information, from 2015, has another neighbour in its section 8.6: measuring one quantity disturbs another for good, and nothing can undo the disturbance while the result is still being kept. That's a second way of saying that a record that stays makes the step impossible to run backwards.

And Morgan Rogers, in his papers of 2019 and 2021 on what mathematicians call toposes of monoid actions, gives the general home for one more way to see it: if every move can be undone by some other sequence of moves, you can never enter a room you're not allowed to leave, because the undoing sequence would have to take you back out. On a finite set, a move where no two arrows meet can always be undone that way: repeat it enough times and everybody comes home.

So the counting isn't mine. It had homes in physics and in automata long before me, and a physical price on it a few years before me. What I do with it starts in the next chapter.

2.3 The hotel that never runs out

Now the way out. If the chairs run out, give the room more chairs. Endlessly many.

Take a strip of paper that never ends. Each move writes a 1 in the next empty box. Done means "there's at least one 1 on the strip." Check it against everything so far. Done stays done: once there's a 1, there's always a 1. The first move takes the empty strip to done. And no two arrows ever meet: two different strips always become two different strips, because you can always erase the last 1 and get back where you were. So here's a machine that settles something, keeps it, and never merges anything.

It got out of the chairs. It didn't get out of paying. Look at what every move does: it writes in a box nobody has used before. The first move does it, and so does every move after that, forever. That's the other column from the last chapter: fresh paper on every tick.

If the endless strip feels like a cheat, it's a famous one. David Hilbert told a story in a lecture course in the winter of 1924 to 1925 about a hotel with endlessly many rooms, every one of them taken, where a new guest still gets a room. Everybody moves from room n to room n + 1, and room 1 is left empty. Nobody shares. The one-to-one move leaves a gap. That's the strip.

The part worth checking is that Hilbert never published it. The hotel got famous because George Gamow put it in a popular book in 1947, and for a long time it wasn't clear Hilbert had told it at all. The historian of science Helge Kragh first guessed that Gamow made it up. Then Tilman Sauer, who had helped edit a volume of Hilbert's lectures, pointed him to Hilbert's lecture notes, which had just been printed in 2013, and in 2014 Kragh took his guess back in a paper called "The True (?) Story of Hilbert's Infinite Hotel." Keep the question mark. Even Kragh's paper puts the lecture in January 1924 in one place and January 1925 in another. Still, this time a story everybody repeated had a real lecture behind it. I've learned not to count on that.

Bring it back to the chairs. In the closed room the person from the door had nowhere to sit because the chairs ran out. In Hilbert's hotel they never do. So the whole argument was about running out of room, all along.

You can run the chairs backwards to see exactly how much room. Take a move that keeps done states done, settles one new state, and never lands two arrows on one place. Everybody already in the done room moves to a different done chair. The newcomer gets a done chair too, and nobody already inside is moving into it, because that would be two arrows meeting. So everybody already inside got seated in the done room with at least one chair to spare. With n chairs, that's n people into n - 1 chairs, which can't be done. So a move like that needs a done room with no end to it. Give the room any fixed size, and the move merges.

That's Hilbert's front desk run backwards. It's also exactly how Richard Dedekind defined "infinite" in 1888: a collection is infinite when you can match it one-to-one with a part of itself that leaves something out. So taking fresh paper isn't a way out of paying. It's the other way to pay, and only a machine with endlessly many states can keep paying it.

2.4 Cut the strip

Now cut the strip to eight boxes. Same rule: fill the next empty box from the left, never touch the earlier marks. There are nine possible strips, from empty to full. Write them down:

0 marks	`00000000`	next: `10000000`
1 mark	`10000000`	next: `11000000`
⋮	⋮	⋮
7 marks	`11111110`	next: `11111111`
8 marks	`11111111`	next: ?

After eight moves the strip is full. What should the ninth move do? Try each answer.

It stays full: then the strip with seven marks and the full strip both go to the full strip. Two arrows, one landing. The chairs.

It stops: then it has no next state, and that's the stopping hole from the last chapter.

It clears the strip: then done went back to not done.

Every answer gives up one of the things the chairs needed. None of them gets past the chairs.

Keeping the whole history like this is the trick Charles Bennett used in 1973, when he showed that any computation can be done without two arrows ever meeting, if you let it keep a copy of everything it did. Rolf Landauer, back in 1961, had already written that for a machine that runs a program that doesn't stop, the space to hold every step "cannot be there." Keeping the history is a real way to pay. It costs a memory that never fills, and every machine you can buy has a memory that fills.

Bennett wasn't the first there, or the last. Yves Lecerf showed in a short note to the French Academy of Sciences in 1963 that machines with no merging can imitate any Turing machine, and hardly anybody noticed. Tommaso Toffoli in 1980, and Edward Fredkin and Toffoli together in 1982, both then at MIT, built logic gates and circuits that compute anything and lose nothing, by carrying extra inputs in and extra outputs out. Their basic gate wasn't new even then: Landauer's own 1961 paper writes one down, calls it reversible and able to do an AND, and doubts anything like it could be useful. What Toffoli added was the proof that it computes everything. And Kenichi Morita proved in 1996 that even a tiny machine with two counters can compute everything with no two arrows ever meeting. Every one of them keeps what it doesn't forget somewhere: a history, extra outputs, a counter that codes the past. They all pay in fresh paper.

2.5 How long can a finite machine hold off?

One more question, because the 5a and 5b example on page one hinted at it. If the machine has a few spare done states, can it put off the merge for a while? Yes, and you can count exactly how long.

At three o'clock a screen says 42. The program is finished. Leave the camera running. At 3:01 it says 42. At 3:02 it says 42. The screen hasn't changed, but

the box behind it still has a next state every tick. It can stay exactly where it is, or something inside it can move while the screen stays put: picture a wheel turning from "first holding position" to "second holding position."

Say the box has K different ways of holding the answer 42. Start one tick before it finishes. Suppose the run goes through its holding ticks without two arrows ever meeting along the way. Could two of those holding ticks be the exact same state? If they were, follow the arrows backwards from both, one tick at a time. Since no two arrows meet along the way, being on the same state now means they were on the same state a tick ago. Keep walking both back. Eventually the earlier one reaches the tick before it finished, the state that wasn't done yet, while the later one is still on a done state. A state that isn't done would have to be the same state as one that is: impossible. So every holding tick needs its own state, and with K of them, it can last at most K ticks. Within K + 1 ticks of finishing, two arrows meet.

You can build one that lasts exactly that long: the unfinished state, then K holding states in a row, the last one stepping to itself. The answer holds for K ticks with nothing merging, and on the next tick the last two arrows meet. A bigger memory puts off the merge. It doesn't make it go away.

2.6 What's being counted

One thing to keep straight before going on. Whether a move merges depends on what you count as the state. The endless strip keeps its whole history in the state, so nothing ever merges; cut it to eight boxes and the same rule merges. Keep a list of every earlier state alongside the current one, and any machine's moves become one-to-one, because two different histories never look the same; a later chapter proves that. Keep only the state one step back, and it isn't enough: two states that differ only in what came before them still land together.

So "this move merges" is always a fact about a move on a stated set of states. Every merge in this book is counted on the whole state the machine says it has, and nothing outside that state is counted.

What I've got now is counting and nothing else: on a finite machine, settling something for good means two arrows meet. The next chapter is about what I do with that, and which parts of it are my choice.

CHAPTER 3

Why charge for it

Everything so far has been counting, and the counting isn't mine. On a finite machine, settling something for good means two arrows meet somewhere. On an endless one, it can mean fresh paper forever instead. Nobody gets to choose that. You drew it.

Page one also said something I do get to choose: I charge for the move where the answer goes from not done to done. This chapter is about how much of a choice that is. If it's nothing but my preference, then the whole machine is a long way of telling you my favourite colour, and you should treat it that way. So I want to find out what it rests on.

3.1 A rule you can refuse

I'm not going to ask you to believe a price. I'm going to write one down as a rule, in the smallest form I can, and you can accept it or refuse it.

Any move where two arrows meet costs at least one.

That's all it says. It doesn't mention done or not done. It doesn't mention checking or settling. It puts a price on one thing: a move that sends two different states to the same place.

It isn't my rule. Charles Bennett, who spent decades working out what computing has to cost, put it this way in 2003: what has to cost is "the erasure of a bit or the merging of two computation paths." Two arrows meeting is two paths merging. I'm taking his rule as it stands and writing it on the move.

Now put it next to the chairs. The chairs said: on a finite machine, a move that settles something for good is a move where two arrows meet. The rule says: a move where two arrows meet costs at least one. Put those two sentences side by side and read them together. A move that settles something for good costs at least one.

That's the rule from page one, the one I said was my choice. I call it the toll. That was my word for it before I could prove any of this: there had to be a toll, a cost in the transition, in the step itself. And look at where it came from just now. Nobody handed it to the machine. It fell out of the chairs and Bennett's rule together.

So it isn't only my preference. But it isn't free of choices either. Count what went into it. There are three, and you can refuse each one.

The done states have to be finitely many, so the chairs run out.

Done has to stay done.

And two arrows meeting has to cost something.

Accept all three and you don't get to refuse the toll. It comes with them. Refuse any one, and the toll goes back to being my choice. So the question "did I have to pick it?" has turned into three smaller questions, and those are better things to argue about. For each one there's a machine that walks out through that door, and you should see each one, because a rule you can't picture breaking is a rule you're taking on faith.

3.2 The first door: endless paper

You've already seen this machine. It's the strip from the last chapter that never runs out. Each move writes a 1 in the next empty box. Done stays done, the first move settles it, and no two arrows ever meet.

So Bennett's rule never charges it anything. Two arrows never meet, so there's nothing to charge. The strip settles its answer at no cost on the price list.

But it didn't get out of paying. It pays in fresh paper, on every move, forever. A price list that only charges for arrows meeting just isn't looking at that column. And on any real machine, the paper runs out, and the chairs come back.

3.3 The second door: let it come back down

Take one coin. The only move is: turn it over. Heads is done, tails is not done.

Draw the arrows. Tails goes to heads. Heads goes to tails. Two dots, two arrows, and they never meet. So Bennett's rule never charges it.

And it does settle something: tails goes to heads, not done to done. At no cost.

What it broke is the second condition. The same move that took tails to heads takes heads back to tails. Done doesn't stay done. So it's not keeping anything. It's flipping a switch back and forth, and the page-one definition already throws it out: a Thiele machine's done never goes back.

Here's the part of this I found sharpest. Forget about done staying done entirely, and keep only the finite chairs. Take any move that takes some state from not done to done. Now go through the done states, one at a time, and ask: does this move send any of them back to not done?

If it does, you've found the coin: the move takes something back somewhere.

If it doesn't, then this move keeps every done state done. That's all the chairs needed, for this one move. So two arrows meet.

There's no third answer. On a finite machine, a move that settles something either has two arrows meet, or it takes something back somewhere else. A move that settles for free and never takes anything back doesn't exist.

That's narrower than "every record costs." It's still true, and I'll take it.

One more picture, so you don't think it's tidier than it is. Three cards, T0, T1 and T2, with a star on T1. The move sends T0 to T1, T1 to T2, and T2 to T1. Look at what it does. T0 gains the star: something settled. T1 loses the star: something taken back. And T0 and T2 both land on T1: two arrows meet. So Bennett's rule charges this move, and yet it doesn't keep anything for good. Charged moves and permanent moves aren't the same list. What's true is one direction: on a finite machine, every move that settles something for good is a move Bennett's rule charges.

And there's an exact way to say which moves Bennett's rule charges no matter what the rest of the price list looks like. A move where two arrows meet gets charged by every price list that follows the rule; that's what the rule says. A move where they never meet doesn't have to be: price that one move at zero and every other move at one, and the rule is still followed. So the moves the rule forces you to charge are exactly the moves where two arrows meet. That one holds on any machine, finite or not.

3.4 The third door: don't charge for it

Take a sheet of paper. It's either blank or stamped. One move: stamp it. Blank goes to stamped, and stamped stays stamped.

Check the conditions. Two states, so the chairs run out. Stamped stays stamped. Blank goes to stamped, so something settles. And draw the arrows: blank goes to stamped, stamped goes to stamped. Two arrows meet. Everything the chairs needed is here.

Now set the price of the stamp to zero.

Nothing in counting stops you. The stamp merges two states, and you just decided merges are free. The answer settled, and the bill says zero.

Price the stamp at one instead, and every condition holds at once, so the three can live together. That stamp at one is the smallest Thiele machine there is, and it gets its own chapter. But the third door asks whether they have to live together, and counting can't say so. The only thing that could close this door is a claim about the world: that two arrows meeting really does cost something, out there, in a real device.

The nearest such claim is Rolf Landauer's, from 1961: that forgetting gives off heat. It's measured for a few devices and argued over for everything else, and it gets its own chapter, with the people who doubt it named. I'll say one thing about it now, because it matters here. Bennett's rule as I wrote it charges every move where two

arrows meet, whether or not you already knew which state you were in. Landauer's heat, as that chapter will show, is only owed for what you didn't know. The rule I'm using is the worst case on purpose: a move that has to work whatever it's handed pays for the worst it could do.

So that's where the toll stands. Accept finite chairs, done stays done, and a price on two arrows meeting, and the toll isn't a choice; it's what the three say together. Each of the three is a choice, and I've shown you a machine walking out through each door. The physics that would close the third door is a premise, and I'll always call it one.

3.5 How much

The toll says at least one. Counting can say more, if you let it.

Take four cards and write A, B, C and DONE on them. Pick one without showing me. Now cover its face with a label that says DONE. Whatever you picked, the face now says DONE. Before the label, how many yes-or-no questions would it take me to find out which card you had? Two: "Is it A or B?" and then one more. After the label, I don't need to ask anything, because there's only one thing the face can say. Two questions' worth of difference got squeezed out.

That's four states landing on one. Each question you no longer need halves the number of places you could have been: four to two, two to one. Two halvings.

If you're willing to go one step further than Bennett's rule, and say each unit of cost pays for at most one halving, then that move costs at least two. And two is enough. In general: if m states were already done and the move settles k more, all m + k of them land on the m done ones, and the move has to pay at least as many halvings as it takes to get from m + k down to m. One newcomer next to one done state is one halving: the toll again.

(The label can be peeled off, so the cards only show the squeeze on what the face says. Whether anything physical got erased is a different question, for the heat chapter.)

There's a plainer way to say what the halving rule asks for. Look at one move and find the place where the most arrows land. Call that its worst pile-up. A cost of one allows piles of two. A cost of two allows piles of four. A pile of three already needs two. That's why the m + k states in the done room had to pay at least that much: spread m + k arrows across m chairs as evenly as you like, and the fullest chair still gets at least (m + k) / m of them. So the rule asks every move to pay for the worst it could ever do, whichever state it's actually handed.

That worst case has a home in physics. Philippe Faist, Frédéric Dupuis, Jonathan Oppenheim and Renato Renner worked out in 2015 the least work any logical step needs when it has to succeed every time, on every input, with no averaging. It

comes to exactly that logarithm of the worst pile-up. An AND gate sends three of its four inputs to 0, so it needs about 1.6 halvings' worth, two in whole units. My rule counts the same thing in units of cost. Joules don't come into it.

You can also count the whole run this way. Write N different starting states on cards. Run each card through the rule once, and stack the cards that land on the same state into one pile. Do it again with the piles. Each tick, count how many different states you're down to. The drops add up exactly: what you started with equals everything lost along the way plus what's left at the end. If you end up with fewer than you started with, some tick had two arrows meet. Turn the cards over first, though. Two runs can show the same answer on the front and still be different underneath, and those are different states.

A word about the k in all this. Physicists write kT for heat at a temperature, and when the counting meets the physics in the heat chapter you'll see k with a little B hung under it, for Boltzmann. That letter has a story I didn't expect. Boltzmann's grave in Vienna has $S = k \log W$ carved on it: entropy is a constant times the logarithm of the number of ways the parts could be arranged. The idea is his. But the constant, and the equation written that way, came from Max Planck, around 1900. In his Nobel lecture in 1920 Planck said the constant "is frequently termed Boltzmann's constant, although to the best of my knowledge Boltzmann himself never introduced it." So the most famous constant in this corner of physics wears the name of a man who never wrote it down. Take the constant away and what's left is the logarithm of a count, which is a number of halvings, which is what the cards just gave you. The constant is the exchange rate between halvings and heat. Whether the world honours that rate is the premise.

3.6 Found, assumed, chosen

That's the place to sort everything so far into three piles, because the rest of the book depends on where each thing falls.

I see it like this: I didn't invent a machine. I found one. That walks straight into one of the oldest fights in mathematics, so here's a short trip into it. G. H. Hardy, the Cambridge number theorist who wrote A Mathematician's Apology in 1940, came down on the found side about as hard as anyone ever has: he wrote that mathematical reality lies outside the people who study it, and the theorems they like to call their creations are really notes on what they saw. The German mathematician Leopold Kronecker supposedly split the difference: God made the whole numbers, and everything else is the work of people. Supposedly, because push on that line and it gets thin fast. It comes down through Heinrich Weber, in a memorial piece written after Kronecker died, pointing back to a lecture from 1886, and that's the only source I could find anybody citing. It's a great line with one witness.

None of that settles whether I found the Thiele Machine or made it up, and a proof wouldn't settle it either. So it's a conviction, and I'll call it one. What it changes is how I hold the thing: like something that was already there, which means I can be wrong about its shape and the shape won't care. And "found" only covers what counting forced. Here are the three piles.

Found: what counting forces on any machine.

Every step of a machine with one next state for each state lands somewhere new, circles back to its start, or has two arrows meet, and after it comes back it repeats forever. You drew that in chapter one.

A run that settles something and keeps it can't circle back to its start, so it has two arrows meet or takes fresh states forever. When only finitely many states are done, the move that settles it has two arrows meet. You sat people in chairs for that.

A machine that stops, with no next state, slips past all of it, and gets caught the moment you say what it does on the next tick.

Assumed: what the found pile is taken to stand on in the world.

That two arrows meeting in a real device, when the device really could have been in either state, gives off heat. That's Landauer's principle, and it's an assumption here, never proved, with its critics named in its chapter.

That a real device can only tell apart finitely many states in any bounded space, so its fresh paper runs out. Nobody has measured where that ceiling is.

That the machine starts clean: if done means "this happened," that's only because it started at not done. That start is put in by hand.

Chosen: what I add, each of which could have gone another way.

To put the price inside the step, as part of what a step is.

That done stays done. That's in the definition because I put it there.

The toll: at least one on the move that settles something. With finite chairs and done staying done, it follows from a price on two arrows meeting, and that price is a choice too.

Which thing gets to count as settling, and what the price list charges. Nothing I've shown you forces either one.

Everything after this chapter is built on those three piles, and I'll keep saying which pile each piece comes from.

CHAPTER 4

The smallest one

The last chapter put a price on the move that settles something. Before I build the smallest machine that carries that price, there's a question I had to ask first. People have been making marks that other people are supposed to rely on for a very long time. Did any of them put a price on the moment the mark goes on and stays? I wanted to know what they thought they were doing, and what it cost them. So here's what I could reach, and where the search ran out.

4.1 Clay, seals, and seven witnesses

Start a little over five thousand years ago, in the Near East, with people who counted sheep with little clay tokens. To send a count across a distance without the courier changing it, they put the tokens inside a hollow clay ball, pressed the shapes of the tokens into the outside while the clay was soft, and sealed it. The archaeologist Denise Schmandt-Besserat has spent a career on these, and she puts the practice at around 3300 BC; the ball is called a bulla.

You can do a version of it with a cookie tin. Glue a paper seal across the lid and the rim, let it dry, and then try to open the lid without tearing the seal. The bare lid would come straight off. The seal is what turns opening it into something you can see afterwards. A substitute seal could fool you, so the tin shows that somebody disturbed it, not that nobody could ever fake it. The clay ball does the same: the outside shows what's inside, and opening it breaks it. Keep that idea, because it comes back: a record is worth something only if it can't be quietly undone.

Now law, which is where people get serious about who's allowed to say what happened. The book of Deuteronomy, chapter nineteen: nobody is convicted on the word of one witness; it takes "two or three." The Roman jurist Gaius, writing in the second century, says a will the court will honour is one sealed by seven witnesses. Why seven? Gaius doesn't say. The legal historian Henry Maine, in 1861, had an answer: the older Roman will was a mock sale, done in front of five witnesses, a man holding the scales, and the buyer of the estate, seven people in all, and when the ceremony fell away the seven seals kept the count. Maine's is the only account of that I found, so treat it as his.

Try it. Write “paid five dollars, signed A” on a napkin and ask a friend to forge it. It takes them ten seconds. Now have two other people initial it at the moment of payment and each keep a copy. If whoever reads it later compares those copies and trusts the people holding them, changing your napkin alone gets caught. Stolen copies, forged initials or a reader who never compares them can still beat it. The claim didn’t change. What stands behind it did.

Which brings me to seals, and the medieval historian Michael Clanchy, whose book is called From Memory to Written Record. England between the Norman Conquest and the early 1300s was learning to trust writing. Until then it had trusted people who remembered. The lesson was painful: seals were forged, charters were forged, and historians today have a whole craft of telling real charters from fakes because there are so many fakes. A seal is only as good as whoever holds the stamp. In 2011 a company called DigiNotar, which issued the certificates web browsers trust, found out the same thing at the cost of its existence: a mathematically perfect signature is worth exactly what the custody of the key is worth, and intruders had taken over the machines that signed. It’s the same failure nine hundred years apart, and I find that oddly comforting.

And then the obvious next question: if seals could be forged, what caught the forgeries? It wasn’t the seals. The most famous forged record in European history is the Donation of Constantine, a document in which the emperor Constantine supposedly handed rule over Rome and the western empire to the Pope. It was most likely written in the eighth century, some four hundred years after Constantine died, and it was leaned on for centuries. Around 1440 a scholar named Lorenzo Valla took it apart by its Latin: the language belonged to a much later age, and it used words, “satraps” among them, that no Roman of Constantine’s day would have used for his officials. (Valla wasn’t even first. Nicholas of Cusa, a German churchman who later became a cardinal, had already called it a forgery in 1433. I know that from the reference works; I haven’t read his book. Valla’s is the version people remember, which is a pattern that keeps turning up.)

Look at what kind of check that was. The question wasn’t “is the stamp real.” It was “could these words have been written when they say they were.” A record can pass the first and fail the second. That was my first hint that what I kept calling one thing, a record, was doing more than one job.

4.2 Two columns and a stamp

The one I’d never have guessed mattered is bookkeeping. Around 1300, the ledger of a Florentine company called Farolfi, kept at its branch in Provence, shows the earliest complete example anyone has found of a trick merchants were working out: write every transaction twice, once where the money came from and once where

it went, and the two columns have to balance. The treasurers of the city of Genoa kept their ledger for 1340 that way. And in 1494 a friar named Luca Pacioli printed the method in a mathematics book, and it spread.

Do it for one day. Keep two columns of every dollar that moves, and total them at night. Have a friend secretly change one entry in one column. The totals disagree; you catch it. Now have them change the matching entry in both columns. The totals agree, and you've caught nothing. So that's what double entry actually is: a check for slips, which a careful liar walks straight through.

Did Pacioli see that? I can only get at it secondhand. A scholar named Forrester, in a paper on the University of Strathclyde's accounting-history pages, reads him as reporting that in many places the books were taken to the local consul to be stamped before a single entry went in. The stamp is a witness again, from outside. The two columns are a method. The stamp is somebody else agreeing to stand behind the fact that these are the books. Keep those two apart, because one is a habit and the other is a commitment.

Then I found where this got turned into a motto. The Royal Society, founded in London in 1660 and still running, picked as its motto Nullius in verba: take nobody's word for it. Where did they get it? From somebody's words, naturally. It's half a line of the Roman poet Horace, from a letter to his patron, saying he isn't bound to swear by the words of any master. So the society that takes nobody's word for it took its motto from a poet and cut the line to fit. That's not a contradiction. It's a nice picture of the method: borrow what's useful, say whose it was, and don't swear by it.

The historian Steven Shapin wrote about what that motto meant in practice, and the answer is more interesting than the slogan. Robert Boyle, pumping the air out of a glass globe to see what happens to a candle or a bird, had three problems: the machine, the report, and the witnesses. He solved the third by having gentlemen in the room who'd sign their names, and the second by writing the report so richly that a reader who wasn't there could picture the scene and count as a witness anyway. Shapin calls that virtual witnessing, and he notes, dryly, that the air pump was very expensive, the big science of the 1600s. So there was a cost. It just wasn't a cost of declaring the fact. It was the cost of the instrument and of getting enough people into the room. Hold on to that difference, because the rest of the book hangs on it.

4.3 Closer to now

The rest of the lineage points the same way, so I'll go through it quickly.

Regulators who made stockbrokers keep their records on media that can't be rewritten (the SEC required exactly that, write once and read many, from 1997 until

2022, when it allowed an audit trail instead) were paying for the hardware. The rule put no charge on the act of writing.

Stuart Haber and Scott Stornetta, in 1991, chained time-stamped records together so that changing an old entry breaks every entry after it. Bitcoin's footnotes lead back to them. Try the idea with a running total. Take five entries, 1, 2, 3, 4, 5, and add each to the total so far: 1, 3, 6, 10, 15. Now change the second entry to 3. The totals become 1, 4, 7, 11, 16. The change travels all the way down. Now also lower the third entry to 2, and the last total is 15 again. Simple addition can be fooled by changes that cancel out. A real chain uses a much harder function than addition, and somebody has to keep a trusted copy of the last value to compare against. (Ralph Merkle's hash tree, from his 1979 Stanford thesis, lets one short value stand for a great many records the same way.) What the running total shows you is how each entry depends on the ones before it. It doesn't put a price on writing one.

Proof of work, which Cynthia Dwork and Moni Naor proposed in 1992 to make junk mail expensive, and which Bitcoin later chained, is the one place where a record really does cost something. But what it charges for is a search: how many tries it took to find a lucky number. An empty block and a full one face the same target. The work pays for the search. Nothing extra gets charged for the moment anything becomes settled.

Robert Floyd and Tony Hoare, asking in 1967 and 1969 how you'd prove a program does what it's supposed to, attached claims to points in a program: "whenever you get here, this is true." The claim is backed by a proof, and the proof is paid for somewhere else, off the machine.

The certificates of complexity theory, going back to Stephen Cook in 1971, are answers designed to be quick to check: hand me the certificate and I can confirm the claim fast. Whether they're also hard to find is the famous open question of the field, P versus NP, and nobody knows. Either way, what's bounded is the checking. Nothing is charged for raising a flag.

And from statistics, a ledger that runs the other way. Dean Foster and Robert Stine's alpha-investing, from 2008, runs a long string of tests against a budget of allowed false alarms: every test spends some of the budget, and every test that finds something earns some back. That's a budget for how often you're allowed to be wrong. A reward there doesn't mean the discovery was true.

4.4 Five jobs

Lay them all down together, and what shows through isn't one thing. It's five different jobs the word "record" has been doing, and I'd been letting one word carry all five. Most of them come with a corpse.

Durability: the record can't be quietly undone. The clay ball, the write-once disk and the hash chain do this one. The corpse is the editable log, and the company that kept its debts off the books: Enron hid debt in side companies set up to keep it off the balance sheet, which is how the Justice Department described it in 2002, and its auditor, Arthur Andersen, shredded paper. Andersen was convicted that year, and the Supreme Court threw the conviction out in 2005 over how the jury had been instructed. Notice that this job doesn't need a witness at all. A copy will do, as long as somebody else holds it.

Authenticity: the record came from who it says. Seals, signatures and the consul's stamp do this one. The corpse is the forged charter, and DigiNotar. A perfectly authentic entry can still be erased, and a perfectly durable one can still be a forgery. They're different jobs.

Evidence: enough independent people stand behind the claim that lying gets expensive. Two witnesses, seven seals and Boyle's gentlemen. This one is about how many people put their own risk next to the claim. Paper has nothing to do with it.

Accounting: the record checks itself for slips. The two columns. And the second column isn't an outside witness. It's you, checking you. It catches a dropped digit, and a careful liar gets past it.

Quick checking: a certificate, signature or fingerprint that makes one particular check fast. Checking doesn't have to be cheaper than making, and I once thought it had to. Plenty of records are checked by doing the whole thing over, and that's fine when the thing is small.

Now the thing none of them says. Not one of those jobs puts a price on the moment a record goes on and stays. Proof of work prices a search. Bookkeeping prices a habit. Witnesses and seals price showing up and guarding the stamp. I looked, with everything I had, and that's where it ran out. If somebody priced that moment itself, they're on a shelf I never found.

So I had to try it myself, on the smallest thing I could build.

4.5 Two boxes and a coin

Take a sheet of paper. Draw two boxes and label them NO and YES. NO is not done, YES is done. Put a coin in NO. Draw a line underneath for tally marks; that's where you keep score.

You get one move. Call it the stamp. From NO, the coin slides to YES. From YES, it stays in YES. Every time you make the move, whichever box you're in, you put down one tally mark.

Play. Start in NO, make the move as many times as you like, and look at the tally. Now try to get to YES with no marks at all. You can't, and you can see why: the only way from NO to YES is the stamp, and the stamp makes a mark.

Now cross out the mark in the stamp's rule and play again. You get to YES with no marks, every time. That's how little the mark is. The board doesn't force it. Chapter three is the reason it's there: finite chairs, done stays done, and Bennett's rule. Here it's written on a card.

Draw the arrows for the stamp. NO goes to YES. YES goes to YES. Two arrows, one landing place. It's the same picture as 4 and 5 both going to 5 on page one.

That's a Thiele machine. A thing (where the coin is). A move (the stamp). A way to tell how settled it is (NO or YES), where YES never goes back to NO. A run that actually settles it (start in NO, stamp once). A price on the move (one mark, never a refund). And the toll holds: the move that takes NO to YES costs one.

Is there a smaller one? Try.

One box? Then there's nothing to settle. The coin is always in the same place, so no run ever takes it from not done to done.

Two boxes, then. A move either sends the two boxes to two different boxes, or it sends both to the same one. If it keeps them apart, there are only two ways to do that: leave each where it is, or swap them. Leaving them where they are never settles anything. Swapping them is the coin from the second door: it takes NO to YES, but it also takes YES back to NO, and done has to stay done. So a two-box machine that settles anything has to have a move that sends both boxes to the same box, and since YES has to stay YES, that box is YES. That's the stamp.

So nothing smaller works, and look at what that means. Chapters one and two found that keeping something takes two arrows meeting or endless fresh paper. Here there are only two boxes, so there's no fresh paper to be had, and the arrows meet. The smallest machine there is already has the merge inside it, in plain view. Nobody added it. A fuse that blows, a bead that drops into a dip and stays, a stamp pressed onto a box: each one is two states and one move that sends both to the done side, and each one, with that move priced at one, is this machine.

I'll call it the latch, because that's what it is: something that catches and holds.

4.6 What the tally is and isn't

Why tally marks? Because they're about the oldest receipt there is. The real ones, notched sticks split down the middle, get their own chapter later, fire and all. The tally on your paper is nothing like those. Nobody holds the other half. You keep it, beside the board, and you could rub a mark out and nobody would ever know.

Hold on to that. Of the five jobs, the latch does a small piece of the first one: none of its moves takes YES back to NO. It has no witnesses, no signature, no second column, and nothing in it checks anything. YES means the coin is in YES. That's all it means. Whether a YES can ever be made to mean "this was checked and can be relied on" is a real question, and it's the one the second half of the book is about.

For now the tally is a scorekeeper's receipt: you know what you paid, and nobody else has any reason to believe you.

4.7 Any machine can wear one

Here's something the latch lets you do to any machine at all.

Take any machine, any states, any moves. Pick one state, call it x, that some move lands on. Now build a new machine right beside it: the same states and the same moves, with one extra coin riding along in its own two boxes, NO and YES. The coin starts in NO. The first time a move lands on x, the coin goes to YES, and it never comes back. Price every move at one.

Check it against the definition. The coin never goes back to NO. Start anywhere the move that lands on x can be made, with the coin in NO, and make it: the coin goes to YES. Every move costs one, so the toll holds. It's a Thiele machine.

Now cover the extra coin with your hand and watch the rest. What you see is the machine you started with, move for move. Nothing about it changed. The coin rode along and the old machine never knew.

And with your hand over the coin, you can't tell which box it's in. Let any spot with the coin in NO count as a place to start. Now find a moment where the old machine is sitting on x. Did it just arrive there, which put the coin up? Or did it start there, with the coin still down? Both can happen, and the part you can see looks exactly the same. The coin is the only place the difference lives.

So every machine that has a move at all is what's left of a Thiele machine once you cover the coin. Even a machine with one state and one move that goes nowhere: the move lands on that one state, so the coin goes up, and what you've built is the latch. Turing's machine read through "has it stopped?" doesn't even need the extra coin: chapter one showed it already has a done that stays done, and stopping is already a step where two arrows meet. Both ways, it points the same direction page one did. The machine with the price in it is the general thing. The computer is that machine with the price left out.

4.8 Which question gets charged?

One more puzzle, because it shows why the definition picks one question and charges only that one.

Put a tile on a table and slide it one place to the right. Let that move be free. Now ask a new question: is the tile in the place it just reached? Before the slide, no. After it, yes. If every question had to follow the toll, that slide would have to cost at least one.

Try it with any free move that changes anything. Ask “is the machine where this move just put it?” The answer goes from no to yes. So if the toll applied to every question you could ask, every move that changes anything would cost something, and nothing could ever be free. That’s why the toll belongs to one question, the one the machine says is “how settled,” and not to every question you could think of.

And for that one question, a free move can’t touch it. It can’t take NO to YES, because the toll charges that. It can’t take YES to NO, because done stays done. So a free move leaves the settled question exactly where it found it. That’s the whole reason the free moves and the settling moves come apart so cleanly.

4.9 The cheapest price list

Last thing about the latch. Put three price cards next to a button: zero, one and two. Somewhere in the machine, pressing that button takes NO to YES. Cross out zero; the toll won’t allow it. One works. Two works too. The toll picked the smallest card it could, without forbidding the bigger ones.

Now take a button that never takes NO to YES, from anywhere. Zero works. So go through the buttons: one on each button that can settle something somewhere, zero on each button that never does. That’s the cheapest price list that follows the toll, and there’s only one cheapest. Notice the price stays on the button: the stamp costs one even when you press it on YES and nothing changes. The toll is a floor, not an exact bill.

And it counts. Every time a run goes from NO to YES, that step costs at least one, so a run costs at least as much as the number of times it settled something. On the latch that’s at most once, because YES stays YES. On a switch that can be turned off for free, you could settle it and unsettle it a hundred times, and you’d pay at least a hundred. A latch pays once, and keeps it.

CHAPTER 5

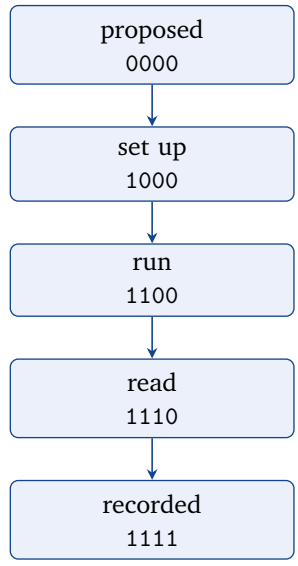
More settled

Up to now everything has had two places: not done and done, NO and YES, the coin in one box or the other. Page one promised more than that. It said settled doesn't have to be yes or no, and to picture a checklist.

That promise matters to me more than it might look. The hunch this whole thing started from was that there's another direction, one ordinary computing doesn't look along. A direction isn't a switch. A switch is just the first line drawn across it. So this chapter takes the switch and finds out what happens when there are many lines.

5.1 A row of cups

Get four cups and a strip of paper. Write five stages of an experiment along the strip, left to right: proposed, set up, run, read, recorded. Agree to do them in that order. Put an upside-down cup between each pair of neighbours. The first cup means "the setup happened," the second "the run happened," the third "the result was read," the fourth "the result was recorded." When you finish a stage, turn its cup right side up. Cups only ever get turned up. None of them ever gets turned back down.



Read down the drawing. Zero means cup down, one means cup up. Four cups give five stages along this run because each new stage turns up one unused cup.

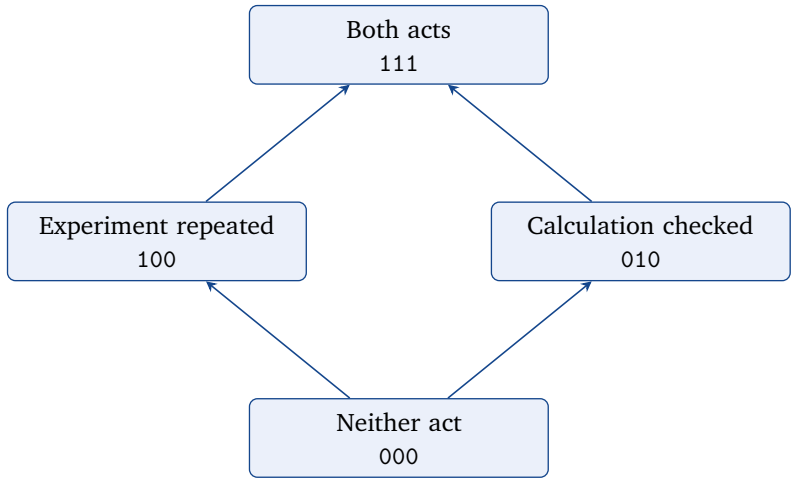
Two things to notice with the cups in front of you.

Each cup on its own is the latch from the last chapter: down or up, and once up, it stays up.

And four cups were enough for five stages. Try it with three. Turn them up one at a time, never back down, and count how many different arrangements you pass through. None up, one up, two up, three up. Four arrangements, and you needed five. You can't do it. Every time the arrangement changes, at least one more cup goes up, and with three cups that can only happen three times. So three cups give you at most four stages, and in general a row of cups that only go up can tell apart at most one more stage than it has cups. A straight line of stages needs one cup fewer than it has stages, and no fewer.

An ordinary binary counter holds a thousand stages in ten bits. That doesn't break anything here: a counter's bits go down as well as up. These cups don't.

Now a harder strip. Some checks don't line up. One person repeats the experiment. Somebody else, separately, works through the calculation. Neither one includes the other. Draw a diamond: "neither" at the bottom, "experiment repeated" on the left, "calculation checked" on the right, "both" at the top. There are two ways up, and the two middle points don't come before or after each other.



Read the cups as repeat, calculation, both. Neither middle point includes the other. The third cup repeats information already supplied by the first two; it is useful for showing thresholds, not needed for storage.

Put a cup on each point except the bottom: one for the repeat, one for the calculation, one for both. Each cup is still a latch. The cups tell you which things have happened. They don't tell you whether the experiment was set up right or the calculation started from the right place. That's a different question, and it comes later.

5.2 Naming what's on the table

Now that you've seen the pieces, here are the names I'll use for them.

The set of places the record can be, whether it's a straight strip or a diamond or something with a hundred points, I'll call the direction.

Each question of the form "has the record reached at least this place?" I'll call a line across it. One line is one cup.

To say "at least this place," you need to be able to compare two places: is this one at or below that one? The strip and the diamond let you see that by looking. Said exactly, it's a test you can put to any two places, and it has to follow two rules. Every place is at or below itself. And if one place is at or below a second, and the second is at or below a third, then the first is at or below the third.

There's a third rule you might expect: if two places are each at or below the other, they're the same place. Every strip and diamond on the table follows it. To see what a rule like that is doing, take it away.

So take two places, left and right, and say each one is at or below the other, without them being the same place. Now there are two cups: "at least left?" and "at least right?" Put the record on left. Both cups are up, because left is at or below

itself and at or below right. Put it on right. Both cups are up again. Look at the cups and you can't tell where the record is.

Now let the record swap between left and right on every step, for free. Is that allowed? It never goes below where it was, since each is at or below the other. No cup ever moves, so nothing ever settles, so the toll never asks for anything. The record moved on every single step, and nothing on the table noticed. It never settles anything either, so it isn't a Thiele machine.

Put the third rule back, and the cups tell you exactly where the record is. Suppose two places leave exactly the same cups up. The cup for the first place is up when the record sits on the first place, so it's up on the second place too, which says the first is at or below the second. The same goes the other way round. Each at or below the other: by the third rule, they're the same place. Without the third rule, the cups can miss moves. Neither is wrong. Dropping the rule just tells the cups not to care about moves between places on the same rung, and they don't.

5.3 The toll on the whole direction

One cup had one rule: the step that turns it up pays. Now there's a row of cups, or a diamond, and one step can turn up several at once. Here's the rule I'd guess first, and it turns out to be the right one: a step that turns up any cup pays at least one. Three cups at once still only has to pay one. The rule asks whether a step turned something up, not how many.

Does that still give you what one cup gave you? Take any run where some cup is up at the end that was down at the start. Somewhere along the run, one step turned that cup up, and that step paid. So the run paid at least one. And every step that turned anything up paid, so the run paid at least once for each of them.

Look at what that argument never used. It didn't care whether the cups sat in a row or a diamond. It didn't care how many cups there were. It didn't even use the fact that cups never come back down: if something could knock a cup back over, the argument still works, because all it needed was the one step where that cup went up.

There's a simple way to tell, from places alone, when a step turns up a cup. Say a step takes the record from place x to place y. The cup at y is up afterwards, because y is at or below itself. Was it down before? It was down exactly when y isn't at or below x. And if y is at or below x, every cup that's up at y was already up at x, so nothing turned up. So a step turns up some cup exactly when it lands somewhere that isn't at or below where it started. That's the toll from page one, written on any direction.

And checking it one cup at a time is the same as checking it on the whole thing. If every cup on its own follows the two-box toll, the whole direction follows the toll, and the other way round.

5.4 One cup isn't enough

Could one cleverer cup do the job of all four? Say you keep one latch and try to read the record's place off the latch together with whatever else the machine shows.

Take the smallest case: a machine with nothing else to show, and a record that climbs through three places, bottom, middle, top. One latch gives you two answers, down or up. Three places, two answers. Some two places get read the same. One cup can't do it.

Try a second guess, closer to how a single cup works. A cup's rule is: the new value is the old value, or the event that turns it up. For a whole row, the obvious copy is: the new place is the lowest place that's at or above both the old place and the event's place. Test it. Take the strip bottom, middle, top, with a cup at middle and a cup at top, and a record that jumps from bottom straight to top in one step, and stays put at middle. With nothing else for the event to look at, the event has to be one fixed place. From bottom, it has to take the record to top, so the event is top. But then from middle, the same rule takes the record to top too, and the record was supposed to stay at middle. The rule breaks.

What works is giving every cup its own event, and letting each cup's event look at where the record already is. A later chapter builds that out over any machine.

How many cups does a direction need? That depends on what you ask the cups to do, so start with the smallest job: tell where the record is, using cups that only ever go up as the record climbs. For that job a cup doesn't have to be "at least this one place." It can go up at several places, as long as climbing never knocks it back down.

A straight strip needs one fewer cup than it has stages; you counted that above. A direction that branches needs at least as many cups as the steps in its longest climb, and at least enough to tell all its places apart.

Put three places above a bottom, none of them above another. The longest climb is one step, but one cup can't tell four places apart, so it takes at least two. Two do it, if you let a cup go up at more than one place: one cup that goes up at the first and third places, one that goes up at the second and third. Every place gets its own pattern. But look at those two cups alone and the third place seems to sit above the first, because every cup that's up at the first is up at the third too. If you also want the cups to show which places sit above which, it takes three: one "at least this place" cup for each of the three.

And sometimes just telling the places apart needs more than both of those. Take two places, each below two others. One step of climb, four places, and two cups still won't do it, because both upper places would have to keep up every cup either lower place has up, and two cups don't leave room for that. It takes three, and showing which sits above which takes four.

5.5 Somebody got here first, from the side

People got here before me, from a direction I didn't expect: programming for many processors at once. Lindsey Kuper and Ryan Newton's LVars, from 2013, are memory cells whose value only climbs, where a program can't ask for the value itself, only whether it has climbed past some point. That's a cup. Two writers running side by side can push the value up in different orders, and a point once passed stays passed. Their programs come out the same every time partly because of which questions the language lets you ask: a program allowed to ask whether the cup is up yet could still see the other writer's timing.

Amin Timany and Lars Birkedal, coming at it from proofs about programs, show in their 2021 paper, as I read it, that any order at all can be carried this way in a logic for proving programs correct, as a value that only climbs and that the running program never reads. That's the closest I've found to somebody else cutting a direction into lines.

The two of them rest on the same shape, a value that only climbs and a question that only asks whether it's past a point, and they use it for opposite jobs. In an LVar the question steers the program. In Timany and Birkedal's logic, the value lives in the proof and the program never looks at it. The shape is theirs first. What I can't find in either is a price on crossing a line.

5.6 Why crossing a line has to cost

Chapters two and three got the toll out of the chairs: on a finite machine, a move that settles something for good lands two states on one. Does that survive a whole row of cups?

The chairs need one yes-or-no question to count with, and now there are many cups. So which cup? The first one I'd reach for is the cup at the place where the record starts. That goes nowhere: the starting state already has that cup up, so there's no newcomer coming in from outside. The cup that works is the one the step turns up, the cup at the place where the record lands. It's up afterwards and it was down before. So put everybody who has that cup up in the done room, and the state the step starts from is the person by the door.

Now it's the chairs again, once for that cup. If the move keeps everybody in the room inside and never lands two of them on one chair, they fill every chair, and there's none left for the newcomer. So one of two things happened: two arrows met, or somebody inside got moved out, which means that cup went back down. If cups never come back down, it has to be the first one. Two arrows meet.

And then Bennett's rule from chapter three does the rest. A move where two arrows meet costs at least one, so a move that turns up a cup costs at least one. The toll on the whole direction rests on the same three things as the toll on one cup: finitely many states with each cup up, cups that never come back down, and a price on two arrows meeting.

Each of those can be knocked out, and each one goes the same way it did for one cup. Let the states be all the whole numbers and let the record count up by one on every step, free: nothing ever merges, the price list has nothing to charge, and every step turns up a new cup. That's the endless strip, paying in fresh paper. Let a two-state record swap back and forth: nothing merges, and the cup it turns up gets knocked back down. That's the coin. Let a two-state record stamp every state up and charge nothing: the arrows meet and nobody asked them to pay. That's the stamp at zero.

So nothing new got smuggled in when one cup became a row, and nothing the one cup needed got left behind. You might expect the diamond to need some extra rule for the two middle cups that don't come before or after each other. It doesn't. Every cup gets its own room of chairs, and the argument in each room is the old one.

CHAPTER 6

Looking isn't keeping

The toll charges the move that settles something for good. But there's another thing a machine does that feels like it ought to cost at least as much, maybe more, and that's finding out. Every time a machine rules out a possibility, it knows more than it did. If two arrows meeting costs something, does learning cost something too?

Most people's gut says yes, and finding out is the expensive part. Most physicists felt that way too, for a long time. It's why one little imaginary creature kept them busy for over a century. This chapter is about where that feeling goes wrong, and where it's right. The short version: looking doesn't have to make two arrows meet. It uses up a blank page instead, and you can hand the page back. Wiping the page to use it again is where two arrows meet. And making a record that stays, one nobody can hand back, is paying for good.

6.1 A small being at a small door

In December 1867 James Clerk Maxwell, who had published his equations for electricity and magnetism two years before, wrote a letter to his friend Peter Tait about a problem he couldn't get rid of. Take a box of gas at one temperature, with a wall down the middle and a tiny door in the wall. Now imagine, he said, a "very observant and neat-fingered being" sitting at the door, who can see the molecules coming. When a fast one comes from the left, it opens the door and lets it through. When a slow one comes from the right, same. Otherwise it keeps the door shut. It never pushes anything. It only opens and closes a frictionless door at the right moments.

After a while the right side is hot and the left side is cold, and you've got a difference you could run an engine off, made out of nothing but looking. That's supposed to be impossible. It's exactly what the second law of thermodynamics says can't happen, and the second law is the one law physicists are least willing to give up. Arthur Eddington wrote in a 1928 book of popular lectures that a theory found to contradict it has nothing for it but to collapse in deepest humiliation.

Maxwell put the being in his textbook in 1871, and it got named Maxwell's demon, though not by Maxwell. William Thomson, later Lord Kelvin, called it a

demon in the journal Nature in 1874. Maxwell, in an undated note to Tait headed "Concerning Demons," tried to take the name back: "Call him no more a demon but a valve." Nobody listened, which is probably for the best, because "Maxwell's valve" wouldn't have lasted a century. But his correction is exactly right, and it's the whole problem: the being does no work. It opens and shuts a door.

So physicists spent the next century trying to find where the being was cheating. The obvious answer pulls hard: the being has to see the molecules, and seeing costs. In 1929 Leo Szilard cut the box down to a single molecule and did the arithmetic. The second law survives, he said, if finding out which side the molecule is on costs at least as much as the being gains. In 1951 the French physicist Léon Brillouin made it concrete: to see a molecule you have to bounce light off it, and, the way the philosopher of physics Jeffrey Bub sums it up, finding out that way costs at least one halving's worth of entropy for every bit. For thirty years that was the answer. Information costs because getting it costs.

But that's a price on one way of looking. It doesn't show that every way of looking has to pay it.

Szilard is worth one more minute, for a reason that has nothing to do with the demon. Four years after the one-molecule engine, in September 1933, by his own later telling, he was waiting at a traffic light on Southampton Row in London. He had just read a newspaper report in which Ernest Rutherford, the physicist who announced the atomic nucleus in 1911, called the hope of getting power out of atoms moonshine. As the light changed, it occurred to him that an element which gave off two neutrons when it swallowed one could keep a reaction going by itself. That's the nuclear chain reaction, thought up crossing a street. It's his memory of himself, and Richard Rhodes's history of the bomb opens with it, in a chapter called "Moonshine," so hold it as the man's own story, not a record made at the time. It changes nothing about the demon. What it adds is a picture of the man: the same mind looked at one molecule in 1929 and one neutron in 1933, and both times the question was what a single small step forces downstream.

6.2 Bennett moves where the cost is

In 1982 Charles Bennett, who had already shown in 1973 that computing can be done with no two arrows meeting if you keep the whole history, took the demon apart one step at a time.

The being looks at a molecule and writes down which side it's on. Draw it. Before the look, there are two ways the world could be: molecule on the left with a blank notepad, or molecule on the right with a blank notepad. After the look, there are two ways: molecule left with the pad saying "left," or molecule right with the pad

saying “right.” Two went in, two came out, and you can read the before off the after. No two arrows met.

But something got used up: the blank page. Before the look, the pad was blank and ready. After it, the pad is full. That’s the fresh paper from chapter one. The look didn’t merge anything. It borrowed a page.

So where’s the cost? The being has one notepad and a lot of molecules. After the first molecule the page is full, and to do the next one it has to wipe it. Draw the wipe. Pad says “left” goes to blank. Pad says “right” goes to blank. Two arrows, one landing place. There it is. Looking feels like the active part, the thing the being does, and wiping feels like housekeeping. The housekeeping is where two arrows meet.

That’s Bennett’s, and it’s the same rule chapter three wrote on the move: what has to cost is “the erasure of a bit or the merging of two computation paths.” In the same 2003 paper he lists copying onto a blank place among the operations that can, in principle, be done without any such cost.

6.3 The demon on a sheet of paper

Build it, because the whole thing is four dots, and you should see the arrows meet or not meet for yourself.

The machine has a hidden bit, which is the molecule’s side, and a display, which is the notepad. Write its four states as pairs, hidden bit first and display second: 00, 01, 10, 11. It has two moves.

MEASURE copies the hidden bit onto the display the way a light switch adds: if the hidden bit is 1, flip the display; if it’s 0, leave it alone. Draw the arrows. 00 goes to 00. 01 goes to 01. 10 goes to 11. 11 goes to 10. Every state has exactly one arrow landing on it. Nothing met.

Look at the two states where the display starts blank, 00 and 10. They go to 00 and 11: the hidden bit got copied onto the blank display. That’s the look.

WIPE sets the display to 0 no matter what it held. 00 goes to 00, and 01 goes to 00 as well. 10 goes to 10, and 11 goes to 10 as well. Two arrows land on 00 and two land on 10. That’s the wipe, and arrows meet twice.

Hidden bit, display before	After MEASURE	After WIPE
00	00	00
01	01	00
10	11	10
11	10	10

So Bennett’s rule charges WIPE at least one, and it’s allowed to charge MEASURE nothing. Zero on the price list doesn’t mean zero spent in a real device; a real MEASURE takes time and energy, and it uses up the blank display. What it doesn’t do is make two arrows meet.

One detail worth seeing. Follow only the two cases you actually set up, the ones with a blank display: after MEASURE they're 00 and 11. WIPE sends those to 00 and 10. Still two different states, because the hidden bit still tells them apart. So on those two cases, WIPE didn't lose anything. The rule charges it anyway, because the rule has to answer for every state it might be handed, and on the full table the arrows meet. The price is the worst case on purpose.

Now be the one watching. You can see the display and not the hidden bit. The display starts blank, so you've got two candidates for what's going on: 00 or 10. One bit you don't know. The machine runs MEASURE, and you look. If the display shows 1, only 10 could have done that. If it shows 0, only 00. Your two candidates went down to one. You learned the hidden bit.

The price was zero. And the machine's own list of where it could be went from 00 and 10 to 00 and 11. Still two. Nothing it could have been got merged with anything else.

You narrowed. It didn't.

6.4 Two rows of dots

That last line is the thing I had to get straight before any of this made sense, so here it is as a picture.

Draw the machine's possible states as a row of dots. Now draw a second row above it: the dots that you, watching from outside, still think it might be on. The first row is a fact about the machine. The second is a fact about you, about what you've seen and what you've crossed off. Both rows can shrink, and they shrink for different reasons.

The machine's row shrinks only when two of its arrows meet. That's what the price charges. Run the halving rule from chapter three along a whole run and the counts just multiply: if the first move costs one, the row can at most halve; if the next costs two, it can shrink to a quarter of that; and so on. So a run can't shrink the machine's own row by more halvings than it paid for, all told. (Count with whole questions instead and you lose a little: four places and three places both take two yes-or-no questions to pin down, so the rounded count can't see that squeeze. The plain count can.)

Your row shrinks when you see something some of your candidates couldn't have shown you, and you cross them off. Nothing happened to the machine. It didn't merge anything. It doesn't know you're watching. It would have done the same thing with nobody there.

The physicist Edwin Jaynes had a name for mixing those two rows up, and it's one of the most useful names I know: the mind projection fallacy, treating what you know about a thing as if it were a property of the thing. Your list of candidates is in

your head. The machine's row is in the machine. The price is about the machine's row, and crossing a candidate off yours doesn't move it.

I have to be fair to Jaynes, though, because he'd argue with where I just took his phrase. In a 1965 paper he wrote that entropy is "an anthropomorphic concept," a phrase he credited to the physicist Eugene Wigner, meaning it belongs to the particular experiments you or I choose to do on a system, not to the system alone. For Jaynes, the second law is about what's known too, and the line I'd like to draw, that physics cares about the machine's row and not yours, is exactly the line he'd refuse. That fight, Jaynes against Boltzmann, is one of the oldest in the subject, and I can't settle it and don't have to. What I need is only this: the two rows are different rows, and only one of them belongs to the machine.

6.5 Who holds the pen

Here's the most beautiful thing in this chapter, and you find it by playing with the four dots, not by thinking about them.

Run MEASURE twice. The first time copies the hidden bit onto the display. The second time adds the hidden bit in again, and adding it twice the light-switch way gets you back where you started. Check the arrows: 10 goes to 11, and 11 goes back to 10. The display went full and then blank again, no two arrows met, and the price was zero both times.

So a display that was blank before the first MEASURE can be put back to blank with no arrows meeting, by a second MEASURE. Not by WIPE. This is the borrowed page being handed back: while the original is still there to check against, the copy can be un-copied.

What's the difference? MEASURE looks at the hidden bit while it clears the display. It has the thing the copy came from, so it can undo the copy. WIPE doesn't look. It sets the display to 0 regardless, and "regardless" is exactly where the arrows meet. Bennett has this in his 2003 paper too: he lists erasing "one of two copies of data known to be identical" as a step that loses nothing, because you're only removing a duplicate. And Lídia del Rio and her colleagues argued in 2011 that the cost of erasing a system depends on how much the eraser already knows about it, dropping to zero, and in the quantum case below zero. The cost of clearing a record falls on whoever clears it, and it's lower the more they know.

Try it with a card under a cup, red or blue, and a note that's blank or marked. Start with the note blank. Lift the cup: for red, flip the note, blank to marked or marked to blank; for blue, leave it. Now the note tells you the colour. Do the same thing again while the card is still there, and the note goes back to blank. Check all four card-and-note combinations and you'll see this rule undoes itself, so no two combinations land together. Now erase the note regardless of the card: for either

card, blank and marked both land on blank. Arrows meet. The cards don't measure anything about heat. What they show is which information each rule gets to use.

6.6 Learning after the first look

A fair objection: the demon's watcher learned something by looking, but any watcher learns something on the very first look, before the machine does anything. So the better question is, after that first look, can a step teach you more, at no cost?

Yes. The smallest machine that does it has three states in a ring: U goes to V, V goes to W, W goes back to U. Say all you can see is one thing: whether it's on W. Start with two candidates, U or V. Neither is on W, so the first look tells you nothing. Take one step. From U you'd be on V, not W; from V you'd be on W. One step told them apart, and a ring never has two arrows meeting, so the price can be zero.

Two states can't do it, ever, with any arrows. If you can't tell two states apart at the first look, what you can see is the same on both, and those two are all the states there are. So what you can see is the same everywhere, forever. Try to draw a counterexample with two dots. You'll draw the same two dots a few times and give up, and giving up is the proof. Three is the smallest number of states where two that look alike at first can come apart later.

6.7 Finishing together

One more count before the big picture, because it's about how much room learning needs.

Run a machine from several different starts, and photograph its whole state at every tick. Lay the photographs out in rows, one row per start, one column per tick. Say every run finishes for the first time at the same tick. And say that in every column up to the one just before the finish, no two states step to the same state. That includes the last step, the one onto the finish: if two runs sat on two different states just before the finish, they can't both step onto one finished state.

Could two photographs on that sheet show the same state? Two in the same column: walk both back a tick at a time, and since no arrows met, they were equal the tick before, all the way back to the starts. But the starts were different. Two in different columns: give them both the same remaining steps. The one that's ahead would finish first, and it was supposed to finish at the same tick as everybody else. So every photograph shows a different state. With N starts and T ticks, the machine needs room for N times (T + 1) states. Running in step and finishing together has a size.

6.8 The certainty count

Now I can put the whole path from not knowing to knowing on one page.

You don't know which state the machine started in. There are a few it could have been, each with its chance. Run the machine. Take any two of those possible starts and watch where they go. Only two things can happen. They land on the same state, and the difference between them is gone from the machine: two arrows met, and under the halving rule from chapter three the price covered it, at most one halving per unit. Or they land on two different states, and the difference is still sitting in the machine, kept as two states held apart. There's no third thing a pair of starts can do.

So everything you didn't know about the start splits cleanly in two: the part the run destroyed, which was paid for, and the part the end state still holds.

Now put a watcher outside, who can see some part of the machine and not the rest. Whatever they see depends only on where the machine started, so whatever they learn is a fact about the start. And the same split holds for it: what they learned is either still held in the machine's end state, or it's gone from the machine and two arrows meeting paid for losing it. A run that costs nothing leaves everything the watcher learned still held in the machine.

That's the exact sense of "looking borrows." The watcher's copy is a copy, and the machine still holds the original. When the machine lets go of the original, two arrows meet, and the bill comes due.

What that doesn't say: how much heat. That's for the heat chapter. It doesn't say the watcher's own notes are free; they need a blank page too. And if the start is known for certain, there's nothing to learn and nothing destroyed, so there's no floor from any of this at all.

6.9 Who gets charged is partly bookkeeping

The physicists had this freedom on the books before me. Paul Fahn in 1996, doing the accounting of Szilard's one-molecule engine step by step, argued that whether you charge the looking step or the wiping step is partly a choice of how you keep the books, and that the total is what's fixed. Takahiro Sagawa and Masahito Ueda in 2009 worked out a bound on that total, with the split depending on the memory. So "a price list could charge MEASURE instead" is a known freedom, not a loophole I found.

The other side is alive too. Ruth Kastner argued in 2025 that Brillouin was essentially right after all: put the real physics of measuring back in, and it's the measuring that pays and stops the demon, and Bennett's free copy fails as physics. And John Norton argued in 2013 that at the scale of molecules, jiggling from heat keeps any process that's supposed to be perfectly reversible from ever finishing, and

finishing it costs more than Landauer's amount. If either of them is right, a real demon's look has a cost after all.

What's on the page here survives that, because it's about price lists: a price list that follows Bennett's rule isn't forced to charge the copy, whatever a real device spends. What wouldn't survive is the physics I've been hanging on it, that a real demon can look for free. If Norton is right, the line between looking and keeping keeps its counting and loses its physics. I can't referee that one.

And in 2010 a group in Tokyo, Shoichi Toyabe, Takahiro Sagawa, Masahito Ueda, Eiro Muneyuki and Masaki Sano, ran a real demon: a tiny particle walked up a staircase by feedback on what a camera saw. The energy it gained was paid for by the information the camera took, and nothing was violated. The demon is real, it works, and it pays. Where it pays is the part that took a century.

6.10 Somebody got there first, goddamn it

I'd been sitting on this idea, that making a result stay so it can't be handed back is what pays, and finding out only borrows, and I thought: this one's mine. Then I found Carlo Rovelli's 2024 paper, "The Thermodynamic Cost of Choosing." A real physicist, with a real paper, saying that making a choice that sticks gives off at least the heat Landauer charges for erasing a bit. He even quotes a German saying, Wahl ist Leiden, to choose is to suffer, and at that point what are you going to do but laugh.

It pissed me off. It's hard to have an original thought in this world. But it didn't last long, because once I actually looked, his was a sentence on the surface, and the machinery under it was different.

On its face he contradicts Bennett. Bennett says measuring doesn't have to cost; Rovelli says any measurement costs. So look at what Rovelli calls a choice. A coin in the air hasn't chosen anything; it has to hit the ground and stop. And then the sentence I'd put on my wall: "stabilization must be irreversible, otherwise it is not stabilization." Bennett's free look is one that can still be undone, which is why it's free. Rovelli's choice is one that's been made to stay, which is why it costs. The way I read them, they're standing on opposite sides of the same line: looking, which borrows, on Bennett's side, and keeping, which pays, on Rovelli's.

That reading is mine, and I owe you exactly how much of it. Push on it with his paper open and it gives in four places. He saves the word "choice" for an outcome that's stable and not predictable, and rules out a thermostat because it runs on a predictable mechanism; the settling move in this book is fixed in advance by a rule, so by his own definition his result doesn't cover it. His choice is one possibility branching into two, which he calls in a sense the opposite of erasing; the toll comes from two arrows meeting, the opposite shape. His stability lasts a while, not forever;

mine is forever, by definition. And he doesn't stand beside Bennett at all: he holds that any measurement costs, credits that to the philosopher Hans Reichenbach, and doesn't cite Bennett. So what Rovelli and I share is a sentence: a choice that sticks costs something. Push on it and his coin hitting the ground, a fact nobody could predict, sits closer to looking than to keeping. I still think it's the same instinct. You've got everything you need to disagree.

And on copying, the nearest physics isn't Rovelli's. It's in biology. Thomas Ouldridge and Pieter Rein ten Wolde worked out in 2017 the basic costs of making a copy of a molecule that stays matched to its template after the two separate, and of destroying one. A companion paper the same year, with Christopher Govern, does the same kind of accounting for how a cell reads out its own sensors, which is a measurement in all but name. The way I read them, the copy that has to stay is the one that pays. Then Jenny Poulton, ten Wolde and Ouldridge in 2019 found the sharp form: in their simple models, when a copy grows with the weakest possible push, its advantage over random copying, and the efficiency of storing that information, both vanish. Pushing harder, with chemical work, makes copies that stay matched after the two let go. That's finding out against keeping, in chemistry, with the cost attached, and biology had the distinction before I did. Theirs is a correlation between molecules, with heat in it. Mine is a reading in a model of computation, with no temperature anywhere.

Two more neighbours have "learning" and "commit" in their titles, so the line here shouldn't be read against them by accident. Sebastian Goldt and Udo Seifert in 2017 did the physics of a neural network learning a rule, and argued that learning has a cost: the entropy the network produces limits how much it can learn. That doesn't contradict this chapter. Their learner isn't copying onto a blank page; it's a driven, noisy physical system, and that's what they price. This chapter only says one way of finding out, a copy onto a blank, is allowed to be free. Nothing here says every way of learning is cheap, and theirs isn't. And from statistics, Aurélien Garivier, Tor Lattimore and Emilie Kaufmann in 2016 compared strategies that explore for a while and then commit to one choice against strategies that keep adjusting as they go. In their setting, over a long run, the best strategies that commit lose about twice as much reward as the best ones that keep adjusting. That's the closest published sentence I've found to "committing costs something" with a number on it. It's lost reward, though, not a count on a ledger, so it's a cousin, not a twin.

6.11 What the toll is for, said at last

Now I can say exactly what the toll charges. Not looking. Not copying. Not finding out. A look can be done with no arrows meeting, on a price list that charges nothing for it; the demon is a machine where that happens. Looking uses up a blank page,

and while the original is still there, the page can be handed back. What gets charged is the one thing chapter two showed can't be done on a finite machine without two arrows meeting: settling something so it stays, a record nothing hands back.

That's what "No Free Insight," the name I gave the toll before I understood it, really covers, and I don't want the name doing work it hasn't earned. Insight in the sense of finding out can come free; it borrows a page. The insight the toll prices is the kind made to stay. Whether that narrower thing deserves the name is fair to argue about. I kept the name because it's the question I started with, and because narrowing it down is the finding.

One more thing this tells you, for later. A later chapter counts what it takes to narrow a list of candidates down to a shorter one, and puts a cost on it. Narrowing alone doesn't force anybody to pay anything; the demon just narrowed two candidates to one for free. So wherever this book puts a price on narrowing, the payment goes in as an assumption, out in the open, and I'll keep calling it one.

CHAPTER 7

Looking through a window

Page one ended on two sheets of paper. One person sorted a list and checked it. The other copied a sorted list off somebody else. Same numbers, same order, and nothing on either sheet tells you which one was checked.

The obvious answer is: look harder. This chapter is about why looking harder doesn't help, exactly when it doesn't, and what does.

7.1 Two boxes under a camera

Put two boxes on a table. Each one has tiles on top, and the tiles spell out the same answer. On the side of one box, a stamp is pressed. On the other, it isn't. Hang a camera straight above them and take a photograph. The two photographs are identical.

Now hand me the photographs and ask which box was stamped. Take a better photograph. Blow it up. Count every tile. None of that gets the side of the box into the picture. The difference is real; it's on the box. It just isn't in what you handed me.

That's the thing I want a word for, and the word is window. A window doesn't see anything. It's a viewing port, an opening that faces one way and narrows what you can take in from where you're standing. The camera above the boxes is one: it faces down, so the tops come through and the sides don't. A display is one. So is a printout, or anything else that lets part of a machine through to you. And behind any one picture there's a pile: every full state of the boxes that would give exactly that picture. The tiles can agree while the boxes differ in whether they were stamped, in what checks were run, in what was erased along the way. All of that sits in the pile behind one photograph. There could be two states in the pile, or a million, or endlessly many.

7.2 A cave, and the people who got good at it

This one is Plato's, from the seventh book of the Republic. Socrates describes prisoners chained in a cave since childhood, facing a wall, with a fire behind them

and people carrying objects along a walkway between the fire and their backs. All the prisoners have ever seen is shadows on the wall. To them, he says, the truth would be nothing but the shadows. Later in the story Socrates reads it out himself: the cave is the world we see, and the climb out is the soul's climb toward what's really there.

The part I need comes a little before that. One prisoner gets freed and taken up into daylight, and he thinks back to the cave. The prisoners, Socrates says, hand out honours to whoever is quickest at spotting the shadows as they pass and best at guessing which comes next. Read that twice. The prisoners get good at it. They build a whole science of shadows, with experts and prizes, and it works, on shadows. What they can't do is say anything about a carried object that leaves no mark on the wall. That isn't because they're stupid. The information isn't on the wall. Skill inside a window doesn't reach past the window, no matter how much skill.

7.3 The shadow of a cube

Try this one tonight with a lamp and a wall. Keep the lamp well back. Hold a cube face-on to the wall and mark the outline of its shadow. Then hold up a square card in its place and move it until the outlines match. Show a friend only the outline. They can't tell you whether the thing had thickness. Ask them, and they'll tell you, truthfully, that they can't know, and that staring at the square won't change it.

Now turn each one a quarter turn. The card's shadow becomes a thin line. The cube's shadow stays a square. Now your friend knows. What changed wasn't how clever they were. It was the window. A second look from a different angle is a bigger window, and what paid for it was the turn.

Edwin Abbott wrote a whole novel about a window like this in 1884: Flatland. The people of Flatland live in a flat world and never see a shape. Put a coin flat on a table and lower your eye to the edge, and the coin turns into a line. That's all the view there is. What gives a Flatlander anything more is fog: a near line is bright, and a line running off into the fog goes dim, so a trained eye can read a triangle or a pentagon off how fast the brightness fades. The training takes time and money, and in the story only people with money get it. The window got bigger, and it was paid for.

Then a sphere passes through the narrator's flat world. What he sees is a line whose ends fade wrong, changing size in a way no flat shape could. He only works out it's a circle by feeling it, and the circle grows from a point to its widest and shrinks back to nothing as the sphere passes through. From one slice he can't put a sphere together. From the slices one after another, seen and felt and argued over, he starts to. The way out of a window is never "think harder." It's always a different window, one you can tell two things apart through when you couldn't through the

old one. Every time, in every story here, that cost something: a turn, a second look, training, time.

Who was Edwin Abbott? Not a mathematician. He was headmaster of the City of London School for twenty-four years, a clergyman, and the author of more than forty books, a Shakespeare grammar among them. He put Flatland out under the name "A Square." His full name was Edwin Abbott Abbott, two A's, and the pen name may be a joke on that, A squared; nice if true, but take it as a lead. The part I like best comes from a sketch of his life on the geometer Thomas Banchoff's pages: Flatland, by far the most read thing he wrote, isn't mentioned at all in his obituary in the London Times. The official record of the man was a window too. Through it you got the headmaster and the grammarian, and not the one book people still read.

A video about that book is where the hunch behind this one started. That story is in the afterword.

7.4 The whole fact

Here's the whole thing, and it's small.

You've got the states, the real things. You've got a window: for each state, it lets through some part of it, and that part is all you get. And you've got a question about the state: give it a state and it hands back an answer. What you want is a reader: something that takes what you can see and gives the right answer to the question, every time.

7.5 When does one exist?

Take two states that give the same picture. The reader sees the same picture for both, so it gives the same answer for both. If the question has different answers on those two states, the reader is wrong on one of them, and there's nothing it can do about it, because it never saw the difference. So a reader can exist only if the question gives the same answer on every state in the same pile.

And the other way round: if the question does give one answer per pile, a reader exists. For each picture, answer whatever the question says about any state behind it; since they all agree, it doesn't matter which one. Existing isn't the same as being easy to find, though. To work out the answer for a picture you still have to find a state behind it, and on some windows that search is hard, or endless.

Try the smallest case by hand. A state is a position, 0 or 1, plus a hidden flag, up or down. Through the window you get only the position. Start at position 0 with the flag down. One move goes to position 1 and puts the flag up. Another move goes to position 1 and leaves the flag down. Through the window, both look like position 1. Now write down every possible reader from "what I see" to "is the flag

up." There are exactly four: from two positions to two answers. Check each one against both results. None gets both right, because both show position 1 and one has the flag up and one doesn't. There's no fifth reader hiding anywhere.

7.6 People who found this on their own

The fact is so plain that it got found separately in every field that has to deal with seeing part of something. Laying them side by side is the kind of evidence I trust: people who never met kept walking into the same wall.

In 1956 Edward Moore, working on the small switching machines that became computer hardware, asked when somebody who can only feed a machine inputs and watch its outputs can tell two of its inside states apart. He made "can't tell apart from outside" precise, and found a sting: even when every pair of states can be told apart by some experiment, there are machines where no single experiment tells you which state you started in. What you can always do is find out which state you're in now.

In 1960 Rudolf Kalman asked the same question for systems that change smoothly, called it observability, and gave a test you can compute. His unobservable states are exactly a pile: states whose difference never reaches the output, however long you watch. Try it with two toy cars on identical roads, a mile apart. Give them the same speed at every moment, and let nothing about the road depend on the mile marker. Their speedometers tell the same story forever. No amount of rereading that story gives you the starting position. A landmark, or a hill at a known spot, would.

Where did Kalman's filter go first? Toward the Moon. In the autumn of 1960 Kalman visited Stanley Schmidt at NASA's Ames Research Center, not knowing what Schmidt's group was working on. They were stuck on how a spacecraft going around the Moon could work out where it was from a few angles its crew could measure through a sight. That's a window problem with a capsule in it: the state is position and speed, the view is a handful of angles, and the job is a running best guess from a string of views. With real work, his method fit, and it went into the studies for navigating Apollo. Leonard McGee and Schmidt, telling the story in 1985, called it a fortunate sequence of events. The filter didn't make the angles show more. It used how the spacecraft moves, plus a string of noisy views, to estimate what the views alone couldn't pin down.

In 1977 Patrick and Radhia Cousot built a whole way of analysing programs on throwing information away on purpose: run the program on rough descriptions of its values instead of the values themselves, and only accept answers that are true for everything the rough description could stand for. If all you know is that a number is between minus one and one, you know its square is between zero and one. You don't know which square. In 1970 Edgar Codd, inventing the relational database,

gave users views, deliberately partial windows onto the stored data, and in 1981 François Bancilhon and Nicolas Spyratos worked out the problem from the other side: if a user changes what they see through a view, which of the many stored states behind it should the database move to? Several are legitimate. That's the pile, seen from the writing side.

Two more fields have it as a founding idea. In the logic of knowledge, what an agent knows is modelled by sorting the possible states into piles it can't tell apart, and it knows a fact exactly when the fact holds everywhere in its pile. The economist Robert Aumann used that in 1976 to prove that two people who start from the same beliefs, and whose estimates are known to each other, can't agree to disagree, and Ronald Fagin, Joseph Halpern, Yoram Moses and Moshe Vardi built a whole logic on the same piles in 1995. And in computer security, Joseph Goguen and José Meseguer defined noninterference in 1982: one group of users doesn't interfere with another when nothing the first group does changes what the second can see. David Parnas, whose cut of a program into modules comes up later, hid things on purpose: a module is defined by the decision it hides from whoever uses it.

There are more. Oliver Kosut and Jörg Kliewer asked in 2018 when a receiver, with a possible attacker on the line and no shared key, can always either get the message right or notice the attacker, and the answer turns on whether the attacker can make what the receiver sees look exactly like an honest message. That's the pile with the probabilities taken out. And in 2022 Anvith Thudi, Hengrui Jia, Ilia Shumailov and Nicolas Papernot asked how a company could prove a trained model had forgotten someone's data, and found the same finished model can come out of different training sets, so the model alone can't show whether a point was used. The only thing you can check is the procedure that was run. Same wall: the finished state is a window, and what happened is in the run.

7.7 Notes passed across a room

Philosophers who study being told things hit the same wall from the other side. Read one after another they blurred together for me, until I built a picture of where each one stands.

Somebody passes you a note across a room. It says the answer to question seven is 42. What crossed the room? Ink. Whatever made the writer believe 42 stayed on their side: the working-out, how careful they were, whether they knew or guessed. Pull that apart and there are five things in play: the claim on the paper, the writer's head, whatever got the writer there, the trip across the room, and your head at the other end. Every position I found puts its weight on one of those five.

The two ends are John Locke and Tyler Burge. Locke, in 1690, says believing something because someone with authority said it isn't knowing it: if the reason

didn't cross the room, you don't have it. Burge, in 1993, says you're entitled to accept what you're told, if it's presented as true and makes sense to you, unless you have a reason not to. I get the pull of Burge; nobody could get through a day checking every note. But a default anybody can trigger by writing something sensible is a record nobody paid for, the thing this book keeps refusing. So on that dial this book sits at Locke's end, and staying there means carrying the check across the room every time.

Jennifer Lackey, in 2008, cuts the wire between the writer's head and what arrives: a teacher who doesn't believe the theory she's required to teach, and students who come away knowing it anyway. Sanford Goldberg, in 2010, puts the weight on the process behind the writer's word, machinery the reader can't see. I'm not comfortable with that, and that discomfort is why this book exists: machinery you can't see is machinery you can't check. Christoph Jäger and Nicholas Shackel built a model in 2025 where the writer knowing is neither needed nor enough for the reader to know, landing where Lackey's teacher did from a completely different direction. Wesley Holliday, in 2009, put trust into a logic of changing your mind, with a graph of whose word each person takes. I've only read a summary of Holliday's paper, so I'm taking that on somebody's word, which is a funny thing to admit in this paragraph. And Michael Burrows, Martín Abadi and Roger Needham, in their 1990 logic of authentication, said: if I believe an authority has the say over a statement, and I believe the authority holds it, I believe it. That's Burge's default with one change: you named the authority before the note arrived.

Kenny Easwaran, in 2009, went after what crosses the room itself. Why do mathematicians accept a long proof, even one partly done by a computer, and turn down a proof based on chance that can be far more certain? His answer is a property he calls transferability. A step-by-step proof puts the reason on the paper: every step crosses the room, so you can learn from the writer without trusting the writer. A proof based on chance leaves the reason behind and sends only the writer's word about what the dice did. That's the cleanest statement I found of what a checked record should do: make the reason cross the room, and run the check on your own side. That reading is mine.

7.8 Slips handed out at desks

People who build systems out of many computers hit the wall too, and one pair measured how far it goes. Picture people at desks, working from one rulebook and passing slips to each other. Nobody sees into anybody's head; all anyone gets is the slips. Somebody may be cheating. What can the honest ones do about it with nothing but slips?

Andreas Haeberlen and Petr Kuznetsov asked in 2009 which kinds of cheating an outside observer can catch at all. As I read them, it depends on whether the cheating ever shows up in a slip. A cheat who breaks the rulebook without changing any slip, or who writes a slip an honest player could have written too, can't be caught by anyone holding the slips. A cheat who fails to send a slip that was due, or sends one the rulebook would never produce, can. The first two are a pile: an honest room and a crooked one handing out the same slips.

Their group's PeerReview system, from 2007, is the ladder up to that wall: every desk keeps a secure log of its slips, and a checker replays the log against a copy of the rulebook. That's close to "show me everything," built. Close, because a log of slips only stands in for a whole desk if the rulebook can rebuild what the desk did from them, and only if the checker's rulebook is the right one.

Then the people asking what you can say once you're at the wall. Ralf Küsters, Tomasz Truderung and Andreas Vogt, in 2010, defined a verdict that names who misbehaved, and showed that being able to check a result is a weaker form of it: one says something went wrong, the other points at a desk. Radha Jagadeesan, Alan Jeffrey, Corin Pitcher and James Riely, in 2009, looked at systems that enforce their rules after the fact, by audit. Robert Künnemann, Ilkan Esiyok and Michael Backes, in 2019, built a way to check it automatically by making it a property of the whole run. And in September 2026 Timo Treitz and Künnemann gave the public logs of web certificates the full treatment, and found that the plain version names a culprit only if the log itself is assumed honest. Somebody always has to be trusted somewhere.

Turn that on me and it bites twice. A raise of the stamp that never reaches the window is the first kind of cheating, the kind nobody outside can catch. And every verdict in this book is "yes" or "no" about one claim; none of them points at a desk.

7.9 Moves that keep to the tabletop

Back to the two boxes. Give them two kinds of move.

Tile moves move the tiles, and only by looking at the tiles. They cost nothing and never touch the stamp. Stamp moves leave the tiles alone and can only press the stamp or leave it as it is. Once pressed, it stays pressed, and pressing it pays the toll.

Now take the stamped box and the unstamped box, with the same tiles, and give them the same tile moves. After one move the tiles still agree, because each next picture depends only on the picture before and the move. After two moves, they still agree. After a hundred. Meanwhile one stamp stays pressed and the other doesn't. You can watch the tabletop for the whole run and never see the difference you started with.

There's one way to break that, and it's worth seeing. Let somebody read the stamp and choose a different tile move because of it. Now the stamp can signal through the tiles. Here both boxes get the same moves, on purpose, to keep the question fixed: what can come through the window, if nothing that reads the stamp is allowed to steer the tiles?

7.10 The clerk with two photographs

Now point the camera at the price, because that's what this chapter is really for.

Say you want to charge for pressing the stamp, exactly: at least one for every move that presses it, nothing for a move that doesn't. And say the person charging is a clerk who only looks through the window: a photograph before each move and a photograph after. The clerk's charge can only depend on those two photographs.

Can the clerk charge exactly? Not if there's a pair like this: two moves that give the clerk the same before-photograph and the same after-photograph, where one pressed the stamp and the other didn't. A stamp pressed inside a closed drawer, next to somebody opening and shutting the drawer. The clerk has one charge for that pair of photographs. If it's at least one, the move that pressed nothing gets overcharged. If it's zero, the press gets undercharged. Nothing read off the photographs gets both right.

The fix is the plain one: put the stamp in the photograph. Then "one when the stamp goes from not pressed to pressed, zero otherwise" works. Showing the stamp isn't the only fix. Any window with no such pair works too; a later chapter has one that never shows where the record is but always shows when it moves.

You can even find the clerk's best careful prices. Put all the moves that give one pair of photographs in a tray. If you never want to miss a press, charge one whenever the tray has even one press in it. If you never want to overcharge, charge one only when every move in the tray is a press. When the tray is all presses or no presses, those two agree. When it's mixed, they can't, and that's the bad pair again. Those are "some member" and "every member" applied to the same tray, which is exactly how the mathematician F. William Lawvere described logic's two quantifiers in 1969, and Robin Cockett, Isabelle Geng, Carlo Maria Scandolo and Priyaa Srinivasan used the same idea in 2023 to stretch measures of resources along a map.

Control engineers have hunted pairs like that for a long time. In 1995 Meera Sampath, Raja Sengupta, Stéphane Lafortune, Demosthenis Teneketzis and Kasim Sinnamohideen asked when an observer can always work out, from what it sees, that a hidden fault has happened. Anooshiravan Saboori and Christoforos Hadjicostis asked in 2007 the question the other way round, for secrets: when can an observer never be sure the system is in a secret state? A colliding pair like the clerk's is one

place where the observer can't tell right away. Their versions are richer than mine. What this chapter adds is the price.

Why can't the clerk just watch the move itself? A clerk who sees the whole state and the move charges exactly with no trouble. So the problem bites an observer who doesn't see the move, or who's handed an account of it they can't trust. If the names of the moves come from a log the program wrote, the program can write whatever names it likes, and a clerk reading that log is reading one more window, one the program painted.

7.11 Look before the concrete covers it

An inspector arrives after the concrete is poured. The question is about the steel underneath. On one job it was sound. On the other it wasn't. Give both jobs exactly the same finished surface, and give the inspector only that.

There's no amount of staring that tells those jobs apart. Hunting for a sound arrangement that could have made this surface doesn't do it either; a sound arrangement was possible on the bad job too. What matters is which one was there. It's the four readers again: one input, two answers needed.

While the steel is still showing, you can inspect it and keep the answer. Or keep enough evidence to inspect later. Either way, something that tells the good job from the bad one has to survive. Wiping some unrelated detail does no harm. Wiping the last difference your question depends on leaves the next inspector with the same surface and two possible pasts.

For one question asked again and again along a run, what you have to keep can be tiny. Start a bit at yes, and at each check, set it to yes only if it was yes and this check passed too. It says exactly whether every check so far passed. A later failure turns it down for good. Put that next to a stamp that says "some check passed at some point" and stays up, and you can see they answer different questions.

7.12 Three ways past the window

Say you're outside, and you want to be convinced a run paid. The window you have doesn't show the count. How bad is that?

Take the smallest setting. A run that presses the stamp and counts one, and a run that only moves the tiles and counts zero, from the same start to the same picture. Call the before-and-after photographs the transcript. Both runs leave the same transcript. Now try to build a checker that reads the transcript and says yes exactly when the count is one. It has to say yes to the first run's transcript. The second run left the very same transcript, so it says yes there too, and the count there is zero. No checker of that transcript is both never fooled and never unfair. Dress

the transcript up with extra parts, and the same thing happens, as long as the extra parts are the same on both.

Economists hit the same wall: Jerry Green and Jean-Jacques Laffont in 1986, and Jesse Bull and Joel Watson in 2007, worked out what a system can decide when people can only claim about themselves what their evidence backs up. An audit can only tell apart the cases the evidence on offer tells apart.

So what works? I know three things, and cryptography seems to use all three. Each gets past the window, not past the cost: the run that pressed the stamp paid either way. And each rests on trusting something.

You can be shown the whole state. Then there's nothing to work out; you read the count straight off. What you trust is that the state you were shown is the state there was.

You can be shown a promise: the ordinary window plus one extra mark that's set when, and only when, the claim holds. Reading that mark works exactly when it can't be set falsely and is always set when it should be. What you trust is whoever can set the mark. A mark anybody can set is just another window. Bruce Schneier and John Kelsey worked out in 1999 how to make a log that an intruder who takes over the machine can't quietly rewrite, and their honest limit is the honest limit here: it protects what was written before the break-in, and nothing after. Mihir Bellare and Bennet Yee had named that property forward integrity in 1997. Point it at an AI agent keeping its own log, and the line moves to the very start: an agent that writes the wrong thing from its first step is past the break-in for the whole log. A 2026 preprint by Laurent Bindschaedler, Quentin Botha and Christoph Siebenbrunner builds exactly such tamper-evident trails for agents and calls it an Agent Flight Recorder.

Or you can ask. The outside party puts a question and gets an answer that pins it down. What you trust is the honesty of whoever answers, because a liar's answer is another shadow. The refined form is Shafi Goldwasser, Silvio Micali and Charles Rackoff's zero-knowledge proofs, from 1985, where a checker is convinced by a conversation built to separate true from false with a guarantee against a cheating prover. Joseph Halpern, Yoram Moses and Mark Tuttle gave them an analysis in terms of knowledge in 1988. My asking, in this chapter, is one round with one fixed question, and nothing in it handles a liar. It's a toy of the real thing, and I'm not going to let the real thing's reputation stand behind the toy.

The real thing even has a cave in it. In a 1989 paper called "How to Explain Zero-Knowledge Protocols to Your Children," Jean-Jacques Quisquater and Louis Guillou, with their families and Tom Berson, told it as Ali Baba's cave: a ring of tunnel with a magic door at the back that opens only for a secret word. Peggy goes in down one side while Victor waits outside; then Victor walks to the fork and shouts which side she should come out of. If she knows the word, she always can. If she

doesn't, she gets away with it only when he happens to call the side she went in. Play it with a coin for Victor's shout. One round, a cheat gets through half the time. Twenty rounds, about one time in a million. And here's the part that belongs in this chapter: Victor ends up convinced, but a video of the whole thing convinces nobody else, because a staged run, where the two agreed beforehand which sides he'd call, makes exactly the same video. The video is a window, and the real run and the fake one cast the same shadow.

What would it take to make that promise-mark binding for real, against somebody who wants to cheat? Manuel Blum's 1981 paper "Coin Flipping by Telephone" is where I'd point. Alice and Bob have just divorced, they live in different cities, and they want to flip a coin over the phone for who gets the car, with nobody both trust to flip it. If Alice flips and announces, Bob has to take her word. If Bob calls first, Alice can lie about how it landed. Blum's way out is for one side to commit first, sending something that fixes the choice without showing it, and open it after the other side calls. What keeps that binding isn't a promise. It's a bet that a certain kind of big number is hard to factor. So in Blum's protocol, binding is earned, on that condition. In this chapter it's only assumed. That's why I won't say these escapes beat a cheater. Andreas Winter, Anderson Nascimento and Hideki Imai worked out in 2003 how fast a noisy channel can carry promises like that, and Claude Crépeau, Rafael Dowsley and Nascimento in 2020 how that rate drops when the one promising can steer the noise.

One more relative: verifiable computation. A box hands back its answer with a short proof that the answer is right, which a checker can verify without redoing the work (Pinocchio, built in 2013 by Bryan Parno, Jon Howell, Craig Gentry and Mariana Raykova), and proof-carrying data passes proofs like that along a whole chain of parties who don't trust each other (Alessandro Chiesa and Eran Tromer, 2010). That's the strongest way out there is, and what it carries out is the claim. The history of which machine ran it, and when, stays behind unless the proof is built to carry it.

Now point all three back at a stamp kept in a machine's own state. That stamp is bits too. Hand somebody a copy of it with a note saying "it's pressed," and they're reading a window: the run that really pressed it and a box built to print that it did cast the same shadow. So keeping the record inside the machine answers "was this earned?" for whoever can see the machine's state and trusts the step that wrote it. For anybody else, it moves the question from "did it run?" to "do I trust the box that says it ran?" Nothing in this book supplies any of the three ways out. The record here lives in one machine, with nobody's signature on it.

7.13 What gets through

Put the two boxes back under the camera. Their tops agree and their sides differ. The photograph answers questions about the tiles. To answer the stamp question, turn the camera far enough to see the side. The whole state was there all along. What changed was which part reached you.

Taking more photographs from the same spot only helps if something different eventually comes into view. When the two runs give the same picture at every step, a longer album keeps the same blind spot. That's Kalman's unobservable states over a whole run.

And none of this is the famous kind of impossibility. Henry Rice proved in 1953 that no program can sort every program by a question about what it does, unless the question gives every program the same answer. That's a deep fact about computing, and a later chapter takes it up. What this chapter shows is smaller and comes first: hand a reader one picture for two states that need two answers, and it's finished before any question of computing comes up. One input, two answers needed.

Which leaves the question I've been pushing ahead of me since the two sheets on page one. Fine: the stamp got pressed, it got paid for, and you can see it through the right window. What does a pressed stamp actually tell you? Anybody can write "CHECKED" on a whiteboard. What would make the stamp mean the list was checked? That's the next chapter.

CHAPTER 8

Established

The last chapter ended at a whiteboard. Give a worker a whiteboard and a pen, and ask for an answer with a line underneath that says “CHECKED = YES.” The worker can write both. So can somebody who copied the answer and checked nothing. The extra line tells you nothing about which one happened.

This chapter is about what it takes for that line to mean something. It’s the case page one called the sharpest, and it’s sharp because the two answers are the same bits, and everything that differs is in what was kept.

8.1 Two computers that end up the same

Put two computers on a table and hand each one the same list of a thousand numbers.

The first one sorts it. It compares, swaps, walks the list again and again until nothing is out of order. Then it does one more thing: it walks the whole list front to back and checks that every number is no bigger than the one after it. That check could have failed. It passes. Then the computer tidies up after itself, wipes its scratch work, and leaves the sorted list in memory. (That check is about order. Whether the list still holds your thousand numbers and no others is a second question, and it would take a second check.)

The second computer never looks at your list. Somebody typed the sorted version in earlier, the same thousand numbers in the same order, and it puts them in memory.

Now wait until both are done and read what’s left: the list, the working registers, where each one is in its program. Arrange for both to stop at the same point in the same program and clear their scratch work, and all of that can agree. The first compared the numbers and checked their order. The second loaded the answer and did neither. Wheel the boxes out, swap them, and bring them back. From what’s left, neither of us can tell which run happened.

Be careful about what that does and doesn’t say. It doesn’t say every atom of two real computers matches; the room, the heat, a clock or a saved log might still carry evidence. And an ordinary computer can keep a “checked” flag or a log that can’t

be changed. These two were set up so the difference got wiped from what you can read. That choice, and what it stops the next reader from answering, is the example.

Who cares? Anybody who's been handed a sheet that says "all tests passed." Anybody who's looked at an inspection sticker on a windshield: the sticker looks the same whether somebody checked the brakes or somebody sold the sticker. An audit, a proof, a ledger, an AI that sounds very sure of itself: each one is a pile of bits that could have come out of either computer, and the question that matters is the one the bits can't answer. Did something establish this, or did it just print it? I'll come back to every one of those at the end of this chapter and say which ones this book actually helps with, and it's fewer than you'd hope.

8.2 Look harder

The obvious answer is: look at everything, not just the end. So make the computers as easy to look at as computers get. Scale them up to the size of a warehouse, every wire a lamp that's lit or dark, every switch something you can walk up to and touch. Nothing is hidden in a chip too small to see. Put them side by side and walk the floor.

While they run, you'd see the difference: one has a whole wing lighting up, comparing and swapping, and the other just copies. But they finish, and the comparing wing goes dark. Walk the floor now and look at every lamp. The sorted list is spelled out in lit and dark lamps in both, the same pattern, and everything else is dark in both, because both cleaned up.

There's no lamp anywhere that says "a check ran here and passed." It isn't hidden, and it isn't scrambled. It just isn't there. Looking harder doesn't help, because the problem was never that you couldn't see enough. Nothing was kept.

8.3 So keep the trace

Then don't let the run go. Keep the trace: every comparison, every swap, the check and its answer, written down as it happens. Wheel the computers out and back, read the traces, and you'll know which one sorted.

It's the right instinct, and it gets most of the way. But it has three problems.

First, a trace is bits, and bits can be typed in. Whoever typed the sorted list into the second computer can type a trace in too: "compared, swapped, checked, passed," a few thousand lines of it, every one of them believable. A trace a program writes is something the program produced. Keeping it doesn't answer the question. It moves it: did something establish this trace, or did it just print it?

Second, the way out of that is to take the pen away from the program. Don't let the program write its own trace. Have something underneath it write the trace, one

step at a time, as the steps actually happen, so nothing a program does can add a line that didn't happen or erase one that did. Airplanes do a version of this. Flight recorders go back to a prototype David Warren built in Melbourne in the 1950s; he was an Australian scientist at the Aeronautical Research Laboratories, working on jet fuel, and after a string of unexplained airliner crashes he wanted a box that would hold the instrument readings and the voices in the cockpit for whoever came looking afterwards. I know his story from museum and encyclopedia accounts, not his own report. What comes out of a recorder is evidence of what its channels carried, as long as the recorder and its sensors worked. A recorded reading can still be wrong about the airplane. The recorder has the right shape, though, and that shape isn't mine. In computer security it's called a reference monitor, and I'll come back to it.

Third, you can't keep everything. If every tick of a run can go either of two ways, then after n ticks there are 2 to the n possible histories, and a fixed number of bits eventually can't tell them all apart. Flight recorders make a practical choice here: a voice recorder loops, replacing the oldest sound when it's full. Thirty minutes, two hours, twenty-five hours: the window depends on the aircraft and the rule, and the FAA's 2026 rule stretches it to twenty-five hours for certain new aircraft. Whatever the length, it's a window.

The fingerprint is the sharpest answer to that, so take it seriously. Fold every new line of the trace into a short scramble of everything before it, and you've got a hash chain. Haber and Stornetta's timestamps work that way, from chapter four; so do the audit logs Bruce Schneier and John Kelsey designed in 1999, and the measurement registers in the security chip a lot of computers carry. That really does keep something about the whole trace in a few dozen bytes. But look at what it keeps: a binding, not the content. To read the history you still need the trace stored somewhere, and the fingerprint only tells you whether a copy somebody hands you is the one that went in. It holds only because nobody knows how to find two traces with the same fingerprint, which is a bet on how hard a search is, not a law of nature. Moni Naor and Guy Rothblum proved a sharp version of that in 2005, and even called it a fingerprint: checking a big file somebody else stores, against an opponent with unlimited computing, costs on the order of the whole file, and doing even slightly better against a weaker opponent needs exactly that kind of bet. And squeezing endlessly growing traces into a fixed number of bits has to land many traces on one fingerprint. That's two arrows meeting, hidden behind how hard it is to find which ones. It's a very clever way of forgetting, not a way out of it.

So the real question was never "save everything." It's: what's the smallest thing worth keeping for good, and what does keeping it take? Not every comparison the sort made. Just the one fact that a check of this very claim passed and the question got closed. And keeping even that one fact for good takes something, on every machine: two arrows meeting, or fresh paper forever. That's chapters one and two.

So it was never true that computers can't keep records. They keep records all day. What nothing in an ordinary computer's rules does is make a record paid for, permanent, and earned. Those come in at different places. Paid for is the toll. Permanent is in the definition: a Thiele machine's settled answer never goes back. Earned, meaning a check of this very claim passed first, isn't in the definition at all. It's what this chapter adds.

8.4 More boxes to tick

Go back to the sorter. It checked the order of the list. Did it also check that none of the numbers went missing? The list can be the same whether that second check ran or not. One lamp saying "checked" won't tell you which work was done.

So put a sheet beside the scratch work, with a box for each check you care about: one for order, one for "nothing missing," and so on. When you come back you want to know which boxes got ticked.

Count what that takes. Say four checks would pass on this list. Each one can be run or skipped. That's two choices, four times over: sixteen different sheets, all sitting behind the same list. Two of them that differ on even one box can't be the same state of the machine, because asking about that box would need two answers. So the machine needs room for all sixteen behind that one list. In general, it's two to the number of checks that would pass, for every list, added up over the lists. And you can build a machine with exactly that much room and no more: keep the list and the sheet, clear the sheet whenever a new list is written, and tick a box only when its check passes.

Look at one small piece of it, because the arrows meet there too. A list that's true or false, and one check. Three states: false; true and unchecked; true and checked. The check takes "true and unchecked" to "true and checked," and leaves "true and checked" where it is. Two arrows, one landing place, right above the same list.

Now keep the old sheets when the list gets wiped and reused. Think of carbon copies filed before each sheet goes in the bin. On each of n rounds you can choose to check or not. Bring every round back to the same final list. Then ask about any round you like: was that one checked? Two histories that disagree on any round need two different states, however alike the final list looks. That's 2 to the n states for n rounds. Keep asking for one more round, with the same freedom and the right to ask about every one, and no finite amount of room is enough. That's where the endless paper comes from: not from keeping one answer, but from keeping a file of answers that never stops growing.

Ask the file a smaller question, "was anything ever checked?", and it becomes a single box that only goes up. That's the kind of record a Thiele machine keeps.

8.5 Computed, stored, established

Three words get used like they're one.

Computed is about the steps. The first computer computed the sorted list. The second didn't.

Stored is about the bits. Both stored it.

Established is neither, and that's the whole problem. It isn't the steps, because they're gone. It isn't the bits, because they're the same. It's this: whoever reads the first computer has a right to lean on "this list is in order," because a test that could have failed was run on this very list, it passed, and then the question got closed. The second computer has the same sentence and no right to it. That's a true fact about the two of them, and it's written down nowhere, in either one.

Four words, and I'll hold them still from here on. A check is the test, the thing that could say no. A commit is the closing: the question shut for good, about that version of that thing. The record is whatever stays behind to say so. And established, in the sense a machine can carry, is all three, about one thing.

What if the second computer, handed the typed-in list, runs the check on it? Then it has what the first had, and if it closes the question and keeps the record, it has established the list is in order. It didn't sort and doesn't need to. The line here is between checked and unchecked, not between worked hard and didn't. Whatever this book counts, it isn't effort.

8.6 The clerk who forgets everything

The picture of a computer in chapter one was a person with a pencil, a strip of paper and a short list of rules, and Turing's 1936 paper takes it a long way. He's careful with the clerk's mind: whatever the clerk knows, beyond a handful of states, has to go down on paper. Then he goes all the way. He imagines the clerk getting up, walking off, forgetting everything, and coming back later. To pick up where he left off, the clerk leaves himself "a note of instructions," and Turing says that note is the clerk's state of mind, as far as the computing goes. Marks on paper and a note are the whole state.

The two computers are sitting right inside that. Suppose the clerk didn't write the note. Somebody slipped it under the door, in the right form, with the right instructions. He comes back, reads it, carries on, and every mark he makes from then on is exactly what he'd have made if he'd written it himself. Turing's model can't tell the clerk who did the work from the clerk who was handed a note saying he did. It was never asked to. Turing wanted something you could check without crawling into anybody's head, and a note on a desk is as checkable as it gets. That's

why the paper still works ninety years later. A Turing machine could keep a note of who wrote the note, and rules about who's allowed to. This one just doesn't.

So the founding picture of computing is a man who leaves his desk, forgets everything he was doing, and trusts a sticky note. I relate to him deeply.

8.7 Information doesn't have it either

The next guess is information: one computer got some, the other didn't. That one dies fast. Claude Shannon, who turned information into a number in 1948, says on his first page that the meaning of a message is "irrelevant to the engineering problem." His measure depends on the chances of the possible messages, and the same message from the same set of chances gets the same number whichever computer it came from, or if one of them got hit by lightning and came out right by luck.

Fine, forget the set of messages and look at the one string. In the 1960s three people separately worked out how to measure the information in one string: Ray Solomonoff, who was after a theory of guessing what comes next; Andrei Kolmogorov, the great Soviet mathematician; and Gregory Chaitin. Their measure is the length of the shortest program that prints the string. The two computers hold the same string and get the same number. And running a program can't add much of that kind of information: what comes out can't hold more than what went in plus the program, give or take a little. Leonid Levin, who studied under Kolmogorov, proved a general version, and has a line for it that's too good not to repeat: you can torture an uninformed witness all you want, you won't get information about the crime.

That doesn't say sorting tells a person nothing. A person in a hurry can read the smallest number straight off the first slot of a sorted list. Short descriptions, useful access, and a record that a particular check ran are three different things.

Logic says the same thing in a different coat. The standard logic of knowing, the one Jaakko Hintikka worked out in 1962, has a property called logical omniscience: if you know some facts, you automatically know everything that follows from them. So by that logic, once you knew every number on your list, you already knew whether it was in order. Hintikka flagged the problem himself, and people have been trying to fix it ever since. The fixes read like a pile until I asked each one the same question: when it lets you write "known" next to "in order," what does it make you write in the margin? Hintikka's margin is empty. Each fix writes one thing there.

The closest one to me is Sergei Artemov's, from 1995. He wanted a logic that could name the proof itself: you only get to know a thing if you've built an actual reason for it, and the reason travels along with the claim. That's almost a record. Almost, because nothing in his margin says the reason was ever run, or when. Others

taught it to count. Ho Ngoc Duc, in 1997, made knowing take steps of reasoning, and Artemov and Roman Kuznets, in 2014, asked how big a reason can get before knowing by it stops being practical. Francisco Bavera and Eduardo Bonelli, in 2018, turned it into a programming language whose terms carry the history of the computation that made them, and Samuel Bucheli, Meghdad Ghari and Thomas Studer, in 2017, put reasons on a time line where they last and change.

Line those up against what this chapter wants in the margin: earned, paid for, permanent, and placed at a step in the run. Artemov's reason is nearly earned. The counted versions put prices on reasoning. The time line gives dates. What none of them has is the closing of a question as a single step with a price on it. Push on mine, though, and it gives twice. Artemov's margin holds the reason itself, and mine only says a check passed. And their prices grow with how hard the thing was to know; this book charges the step that closes the question, and how long the checking took never enters it. For a measure of effort, read theirs.

And yet you know something after the sort that you didn't before. You can stop worrying. Something happened, and it's the first solid clue: whatever established is, it isn't new bits. It's the same bits, settled: a question sorted into yes and no, with yes made to stay.

8.8 Statues that walk off

If it isn't in the information, maybe it's in the knowing, so I went and read the philosophers. They'd been arguing about the difference between having the right answer and knowing it for a couple of thousand years.

Plato's version is a joke about statues. There was a story that the sculptor Daedalus made statues so lifelike they'd walk off if you didn't tie them down. True beliefs, Plato says in the Meno, are like that: they wander off, and they're not worth much until you tie them down with the reason why. Once they're tied down, they're knowledge, and they stay put. Knowledge tied to a reason, and knowledge that stays.

Then in 1963 Edmund Gettier broke the tidy version in three pages. Smith and Jones are up for the same job. Smith has good reason to think Jones will get it, and he's counted the coins in Jones's pocket: ten. So Smith figures, with good reason, that the man who gets the job has ten coins in his pocket. Then Smith gets the job, and, without knowing it, Smith also has ten coins in his pocket. Smith had a belief that was true and backed by a reason, and it still wasn't knowledge. He was right by luck, because his reason was about the wrong pocket.

That's the push that matters for a machine. Checking one pocket and closing the question about another is right by luck. So whatever a machine closes, the check

behind it has to be about this very claim, and here's exactly what that means. Four things.

It checks the same property the commit names, not a cousin of it.

It checks that property on the same thing the commit names, not the thing next to it.

Nothing has written to that thing between the check and the commit, so it's still the same version the check looked at.

And the check is honest: it says yes when the property holds and no when it doesn't, on anything it could ever be handed.

Break the first three and you're Smith with his ten coins. Break the fourth and the check is a rubber stamp. Two notes on those. The fourth asks a little more than it needs: to rule out luck, a check only has to never say yes when the property is false. And the third is a very old bug in computing wearing Gettier's coat. Security people call it time-of-check to time-of-use: check a file, use it, and something swaps the file in between. Matt Bishop and Michael Dilger wrote up how to hunt for it in 1996. Databases guard against the same thing by making sure, just before a commit, that nothing they read has been written since; H. T. Kung and John Robinson called that optimistic concurrency control in 1981. A philosopher's pocket and a programmer's race condition are the same bug. One limit, said now: a machine can only promise "nothing has written to that thing" about its own memory. For brake pads or a voter roll, that condition becomes an assumption about whoever measured.

Does anything fix Gettier for good? People tried for years. The first patch was "no false steps in the reasoning." Then the philosopher Alvin Goldman came back with barns, crediting them to Carl Ginet. You're driving through a county that, without your knowing, is full of fake barns, just painted fronts propped up to look real. You look at the one real barn and think "that's a barn." It's true, you believe it, you have a good reason, your own eyes, and there's no false step anywhere. And you were still lucky: if you'd glanced at the next one over you'd have believed the same thing and been wrong. Every patch gets a new barn. What I keep from that: whether a check was good isn't only about this one case. It's about what the check would have done on the cases next door.

8.9 What "could have failed" means

I keep saying the first computer's check could have failed. The computer is deterministic. Given your list, the check was always going to pass. So in what sense could it have failed?

Here's the sense. Hand the checking step a list that's out of order, and it says no. That's all. Chance has nothing to do with it. A check splits every possible input into a pile it says yes to and a pile it says no to, and "could have failed" means the no

pile isn't empty. A check that says yes to everything has an empty no pile. It tells you nothing, however long it runs. The philosopher Robert Nozick made a version of this a condition for knowing, in 1981: you know a thing only if, had it been false, you wouldn't believe it.

Notice that "could have failed" is about what the check would have done on other lists. It's not a fact about the finished state at all. Look at the first computer's memory: a sorted list. Nothing in those bits says "and whatever made me would have caught a mistake." That's exactly why the only way a machine can own this is to write something down at the moment the check passes, and have that something mean what it says. Even then the record only carries half of it. It can say a check ran on this version and passed. That the check would have said no on a bad list is a fact about the checker, and that half rests on a proof about the checker, made once, outside any run.

Rig a check to pass only on your one list. Is that cheating? No. It's an honest check of a different claim: "this is that exact list." Its no pile is huge: every other list. It establishes exactly that and nothing more. A check establishes what it's honest about.

And one thing I'll say plainly now: nothing here stops you from asking a question that can't come out no, like "is this number at least zero." Ask one, pay for the check and the close, and nothing is established but the record of asking.

8.10 Computing already had a word for it

Computing already has a word for this and uses it every day. Anyone who's used a database has seen it: COMMIT. A database does a pile of tentative work, and then there's one moment, the commit, written to a log, after which the change counts.

Jim Gray, who more or less built the theory behind that, opens his 1981 paper on it by saying the idea comes from contract law, and his example is a wedding. People negotiate, somebody asks, two people say "I do," and from then on it's binding. If it was a mistake, you don't erase it. You do another transaction to fix it, the way a bad contract gets a second contract.

The wedding was a philosopher's example first. J. L. Austin, lecturing at Harvard in 1955, said "I do" makes a marriage; it doesn't describe one, and sayings like that aren't true or false. They can misfire: the wrong person says it, a step gets skipped. A commit works the same way. Writing the commit record doesn't report that a question got closed. Writing it is the closing. It can't be false; it can misfire, by closing without the check, or about the wrong pocket. Then somebody else reads the record, and for them it turns into a report, "this was checked and closed," and a report can be false. That's how a commit gets tied to the truth: through a check, and no further than the check reaches.

Robin Milner, building a proof checker in the 1970s, ran out of memory, so he decided to keep only the finished theorems and throw the proofs away, like a teacher rubbing out the blackboard to make room. The obvious worry is forgery: if the proofs are gone, what stops someone from just writing a theorem down? His answer was beautiful. Theorems got their own type in the programming language, and the only way to make one was through the rules of logic. You couldn't forge one because the language wouldn't let you build it any other way. He named that language ML, for Meta Language. I wanted to call my machine the meta machine. I'm choosing to treat that as a coincidence.

Bitcoin keeps a history that's expensive to replace. A node that comes back online checks the candidate chains against its rules and follows the valid one with the most work behind it. Replacing an old block means redoing the work that ties every later block to it. That's protection under assumptions about the attacker and the network, and a recent block can still be undone. And the work is tied to something scarce, electricity and hardware. A unit of cost in this book is tied to nothing. The economist Eric Budish showed in 2025 what that tie has to cost: for Bitcoin's kind of trust to hold, the honest work has to stay expensive at every moment compared with what an attacker could win. A flat charge of one unit can't do that, and isn't trying to. It's a count, not a lock. The lock is the rule that the close waits for the check, and it would work at a price of zero. The price is there because, on a finite machine, keeping the answer makes two arrows meet.

And Bitcoin wasn't first. Pull on its footnotes and three of the eight references in the paper that started it point to Stuart Haber and Scott Stornetta, who in 1991 worked out how to stamp a document with a time nobody could fake later, by chaining each stamp to the ones before. A company built on their idea, Surety, printed a short fingerprint of its chain in the classified ads of the New York Times once a week from 1995, and was still doing it in 2018, under "Notices & Lost and Found," according to a piece the journalist Daniel Oberhaus wrote that year. To make a record nobody could quietly rewrite, they put it somewhere a million people would keep a copy without meaning to.

All of those work. I lean on all of them. The next thing isn't "they're broken." It's where they sit. The database's commit is bits in a log, kept by a database program, on a machine whose own step will overwrite those bits the moment some instruction says to. Milner's guarantee holds inside programs written in his language and nowhere else. Bitcoin is a set of rules running on ordinary computers that do whatever they're told. Each one is a program or a rule sitting on top of a machine that can't tell the two computers apart.

The obvious answer: put the rule one level lower, in the piece of the system every action has to pass through. Security people call that a reference monitor. James Anderson described it in a 1972 planning study for the US Air Force: a piece every

access passes through, that can't be tampered with, and small enough to check. Fred Schneider argued in 2000 that a monitor that only watches a run as it goes can enforce exactly the rules whose breaking shows up at some point you can name, and "no close without an earlier passing check of the same claim" is one of those. And Arthur Azevedo de Amorim and six coauthors built monitors in 2015 that check a rule at every single instruction, and proved them correct. Putting a rule under every program, so nothing can get around it, is their move, not mine. A monitor that wouldn't let any program raise its record without paying and without the check first would be one way to build what this book describes. That's an implementation, not the thing. What I'm putting under that move is narrower: a record that has to be paid for, that has to be earned, and a reason from counting why the price of a record that never comes down can't be zero on a finite machine.

8.11 The two computers, put back together

Here's the two-computer story again, rebuilt as Thiele machines, with the record and a running count written out. The record here is established: it goes up only right after a check of the claim passes. The prices are one to check, one to raise the record, and nothing for ordinary work.

Why does the check cost anything, if effort never counts? Not for the work. The check leaves a mark saying it passed, so the raise can see it, and you've already drawn what that mark does: "true and unchecked" and "true and checked" both land on "true and checked." Two arrows meet, so Bennett's rule charges it. The raise costs one because that's the toll. And the copier's raise costs one even though it's refused, because the price is on the button, the same as the stamp pressed on YES in chapter four.

step	sorter does	record	count	copier does	record	count
start	nothing yet	no	0	nothing yet	no	0
1	sorts (ordinary steps)	no	0	copies in the typed list	no	0
2	checks "in order" on this list: passes	no	1	(no check)	no	0
3	raises the record	yes	2	tries to raise the record: refused	no	1

At the end, the lists are still the same. Everything else differs. The sorter's record is up and its count reads 2. The copier's record is down, and it paid 1 for the attempt, because an attempt costs what it costs. Nothing the copier can do raises its record without first running a passing check of the claim. And anything that lets through only the list can't tell you which is which. Walk the warehouse now and there's finally a lamp that's lit in one and dark in the other.

How literally should you take that table? It's a picture of the condition, and that condition is the whole of what established means. What it needs from the question is a checker that says yes exactly when the answer is yes. Without one, the record is honest about whatever the checker reads, and only by convention about your list. That's Gettier's pocket again.

And a lamp for whom? The record is bits too. It's a lamp in the machine, for anybody walking that warehouse. Print it out and hand it across a desk, and you're back at the sticker on the windshield: the person reading it has to trust whoever handed it over, or have some way to check the box that says it ran. That's the last chapter's problem, and this book doesn't solve it. What changes is the question, from "did it run?" to "do I trust the box that says it ran?" That's a smaller question. It isn't no question.

So, back to the list from the start of the chapter. The windshield sticker is about whether a piece of paper is genuine, and nothing here touches it. The "all tests passed" sheet fits only if the test is a checker proved to mean what it says; a set of tests that can miss a bug establishes "this procedure said yes," which is true and smaller. An audit or a ledger kept by a program on an ordinary computer is bits the program wrote; it fits only where the step underneath keeps the record and refuses to raise it without the check. A proof is one example: checking its steps is a check that can say no, though whether its statement says what you meant is still yours to read. The AI that sounds sure of itself isn't touched at all.

Now the honest half. Take away the rule that the raise waits for the check, and keep only the bare toll from page one, and the copier has an easier move: pay one, raise the record, done. The toll is satisfied and the receipt is real, but it's a receipt for nothing. The bare toll prices the closing of the door. The check behind it goes unpriced and unrequired. That's why established is a condition you add, not something the definition hands you.

8.12 What putting it in the step buys

The hardest objection is one I'd rather make myself. You defined a mark and a charge, then showed the charge gets paid when the mark goes up. That's your definitions agreeing with each other.

For the bare machine, that's fair, and the copier's receipt for nothing is the proof. What makes the record mean established is the condition that it goes up only after a check of that very claim passed, meeting the first three of the four conditions, with the fourth, an honest checker, a fact about the checker. And the condition isn't about one program. On every run from a clean start, if the record is up, then somewhere earlier a check of the claim passed, with nothing touching the thing between the check and the raise.

So a raised record, from a clean start, tells you a claim went through a passing check before the raise. That piece of history is what it keeps, and only that. It doesn't say the claim still holds at the end; the thing it's about can change after the raise. The record stays up about the version that was checked.

That "every run" is what a rule on every step buys. A log written by one program is only as good as that program. A rule the step itself enforces covers every run, hostile programs included. There are two limits on that "every," and I'll say them now so you don't have to find them later. It's every run, not every everything: from a clean start, one step at a time, on a step that does what its definition says. A hostile start, a soldering iron on the record's bit, or a hostile reader is outside it. And on their own, the record and the count are what people who prove programs correct call ghost state: erase them and every program computes exactly what it did before. What isn't ghost is whatever the step reads before it lets the record rise: the check in front of it.

A computer that runs a Thiele machine, your laptop say, isn't that machine. It implements it, inside bounds, the way a laptop implements a Turing machine. Turing's tape never runs out, and a laptop's memory does; strictly, a laptop is a very big finite machine, and it acts like a Turing machine only on the computations that fit. Nobody minds, because the bound is known and most of what people compute fits. That's the same deal any device running a Thiele machine gets. The bounds aren't fine print. They're the whole difference between running a model and being one.

How much can a difference in size matter? On 4 June 1996 the first Ariane 5 rocket lifted off and broke apart about forty seconds later. The inquiry board, chaired by the mathematician Jacques-Louis Lions, traced it to one conversion in the guidance software: a 64-bit number, a value tied to horizontal speed, squeezed into a 16-bit whole number. On Ariane 4, where the software came from, that value could never get big enough to overflow, because of how that rocket flew. Ariane 5 flew differently. The value got too big, the backup unit failed on it, the main unit failed the same way a fraction of a second later, and the code doing the converting didn't even serve a purpose once the rocket was off the ground; it was kept so the software would match the old rocket's. The board's report says the assumption that made the conversion safe had been agreed on and then, without anyone meaning to, kept out of sight of any outside review. An assumption about size that holds on one machine is a fact about how that machine gets used. The code doesn't carry it, and yet it travels with the code without anybody noticing. The only defence I know is the dull one: write the assumption into every statement that needs it, where the next person will trip over it.